

How to Delete Without a Trace: Certified Deniability in a Quantum World

Alper Çakan*

Vipul Goyal†

Justin Raizes‡

Abstract

Is it possible to comprehensively destroy a piece of quantum information, so that *nothing* is left behind except the memory of whether one had it at one point? For example, various works, most recently Morimae, Poremba, and Yamakawa (TQC '24), show how to construct a signature scheme with certified deletion where a user who deletes a signature on m cannot later produce a signature for m . However, in all of the existing schemes, even after deletion the user is still able keep irrefutable evidence that m was signed, and thus they do not fully capture the spirit of *deletion*.

In this work, we initiate the study of *certified deniability* in order to obtain a more comprehensive notion of deletion. Certified deniability uses a simulation-based security definition, ensuring that any information the user has kept after deletion could have been learned without being given the deleteable object to begin with; meaning that deletion leaves no trace behind! We define and construct two non-interactive primitives that satisfy certified deniability in the quantum random oracle model: signatures and non-interactive zero-knowledge arguments (NIZKs). As a consequence, for example, it is not possible to delete a signature/NIZK and later provide convincing evidence that it used to exist. Notably, our results utilize uniquely quantum phenomena to bypass Pass's (CRYPTO '03) celebrated result showing that deniable NIZKs are impossible even in the random oracle model.

1 Introduction

In their seminal work, Broadbent and Islam [BI20] observed that it is possible to enforce the deletion of secret information via quantum mechanics, introducing the notion of encryption with certified deletion. The notion of certified deletion was subsequently generalized to a wide variety of cryptographic primitives, e.g. [HMNY22, HKM⁺24, Por23, APV23, AKN⁺23, BK23, BGK⁺24, BR24].¹ Following this line of work, Morimae, Poremba, and Yamakawa [MPY24] formally defined the notion of revocable signatures². A revocable signature scheme enables a sender to sign a message m in such a way that it may later verify that the signature has been destroyed, in the sense that the user who previously received the signature can no longer produce a valid signature for m .

We argue that although the existing notion of revocable signatures is well-motivated, it loses some of the spirit of deletion. Their definition requires that after the user produces a valid certificate, they cannot produce a signature that is recognized as valid *by the honest verification algorithm*. On the other hand, the user may be able to use the leftover information to prove to a

*Carnegie Mellon University. acakan@cs.cmu.edu.

†NTT Research & Carnegie Mellon University. vipul@vipulgoyal.org

‡Carnegie Mellon University jraizes@andrew.cmu.edu.

¹The notion of cryptography with certified deletion has also been referred to as “revocable cryptography”.

²We note that while it was not formally studied before [MPY24], it was folklore that revocable signatures follow from public-key quantum money. [MPY24] construct revocable signatures from one-way functions.

third party that the message was signed *at some point*. [MPY24]’s definition does not rule out, or even consider, such an attack. Indeed, all the existing constructions, including that of [MPY24], suffer from such a strategy.³

This crack in the armor of their definition comes from a custom tailoring of the certified deletion paradigm to the specific security guarantees of the target cryptographic primitive - a trait shared by other works on certified deletion. In light of this, we propose a more comprehensive and application independent philosophy for certified deletion:

Once a user produces a certificate of deletion, it should be as if they never received it in the first place.

This philosophy has close connections to a natural notion called *deniability* in cryptography, which has been well-studied in the classical setting [DDN91, CDNO97, DNS98, Pas03] and has recently received attention in the quantum setting [CGV22]. Although there are good solutions for interactive deniability, non-interactive deniability is notoriously hard in the classical setting. Solutions for signatures make heavy compromises such as sacrificing public signature verifiability or requiring well-synchronized global clocks [JSI96, HW21, ABC22, BCG⁺23], while deniable non-interactive zero knowledge is simply impossible even with a random oracle or common reference string [Pas03].

Certified Deniability for Signatures and NIZKs. In this work, we explore the formalization of our new certified deletion philosophy, which we call *certified deniability*. We define and construct certified deniability for two natural non-interactive primitives: signatures and non-interactive zero knowledge (NIZK). In a signature or NIZK with certified deniability, any user may publicly verify that a message was signed or that some statement x is true. However, once a signature/NIZK is proven to be deleted, the party that had it will retain only their memory of whether they decided to accept or reject it. As a consequence, they are unable to convince a skeptical external verifier of the truth of the statement, even if the verifier is willing to accept an alternative proof outside the constraints of the original protocol.

Motivating Applications. As a motivating example, consider two parties Alice and Bob who sign a contract. Their signatures should be publicly verifiable so that if either party does not uphold their end, the contract can be publicly verified in court. However, after the contract has ended, neither party should be able to provide evidence that, e.g. Alice once paid for a particular product or service from Bob. That is, *the contract becomes deniable*. To ensure this, at the termination of the contract, the parties can both provide a certificate of deletion of the other’s signature.

As another example, consider an anonymous whistleblower who sends information to a journalist. The journalist wants to be sure that the info comes from a reputable source, so it should be signed. However, the whistleblower doesn’t want to be implicated if the journalist’s computer is hacked later. To ensure this, they can ask that the journalist deletes the signature after verifying it. For further motivation, we refer the reader to the rich history of work in deniability, e.g. [DDN91, CDNO97, DNS98, Pas03].

One can also imagine signed credentials which, once returned, are erased from receiver’s memory. For example, once an employee leaves secret service, it should no longer be possible for them to even prove that they ever worked in secret services.

³We note that despite this weakness, existing constructions and definitions still have applications in settings where the third-party verifier can be trusted to be honest, such as building access cards.

1.1 Our Results.

As the main contributions of this work, we lay the definitional groundwork for signatures and NIZKs with certified deniability and study what assumptions enable them.

Simulation-Based Deniability. To define certified deniability, we use the simulation paradigm to capture the comprehensive deletion philosophy mentioned before. We require that the state of the adversary after producing a valid deletion certificate can be simulated *without* having received a signature (or NIZK) in the first place. This gives us a guarantee of the form “*anything an adversary could learn from a deleted signature/NIZK, they could learn without it.*”

To achieve this notion for signatures and NIZKs, we augment the Fiat-Shamir transform to add certified deniability in the quantum random oracle model (QROM).

Theorem 1 (Informal; [Theorem 6](#), [Theorem 5](#)). *There exist NIZKs with certified deniability in the QROM. Furthermore, if one-way functions exist, then there exist signatures with certified deniability in the QROM.*

Notably, this result bypasses Pass’s impossibility for deniable NIZKs in the random oracle model. This can be seen as a unique application of quantum mechanics to force a user to “forget” information in a quantifiable way. For more discussion, see the technical overview.

Evidence Against Plain Model Constructions. Since our results are in the QROM, a natural next question is whether we can hope to achieve certified deniability (of either variety) in the *plain* model. Unfortunately, it appears that there are significant barriers to doing so. We show that if the security proof treats the adversary as a black-box, then it cannot hope to show either of the above notions.

Theorem 2 (Informal). *There is no signature/NIZK with certified deniability in the plain model with a security proof that makes black-box use of the adversary.*

Thus, any valid proof of security for a plain-model construction must be non-black-box. Although there have been many improvements in non-black-box techniques in the past decade, e.g. [\[BKP18, BS20\]](#), the technique we use for the barrier is particularly amenable to obfuscation, which has also seen vast improvements recently [\[JLS21, JLS22, JLLW23\]](#). We leave a more definitive answer to the question of certified deniability in the plain model to future work.

2 Technical Overview

Evidence-Collection Strategies. We begin by discussing how an adversarial verifier can retain irrefutable evidence of a signature even after it has been revoked in the existing revocable signature constructions. We will consider MPY24’s construction as an example, though the same approach works for other existing constructions (such as the public-key quantum money based approach). MPY24 begins by defining and constructing a new primitive called *2-tier tokenized signatures*. This primitive enables a simple construction for revocable signatures that can sign only the message “0”. Using Lamport signatures, these can be extended to sign any message, but the key cannot be re-used to sign a second message (“no-query” security).

Finally, they achieve revocation security even when the adversary receives additional signatures (“multi-query” security) by having a single master signing key. When the signer wishes to sign a new message m , they generate a fresh one-time key pair $(\text{sk}_R, \text{vk}_R)$ together with a revocable

signature $|\psi_R\rangle$. Then, the signer signs $\text{vk}_R\|m$ under their global signing key to associate the new key with m .

The problem lies in this final step. Every signature on a message m consists of a *classical* signature σ on $\text{vk}_R\|m$, together with the revocable signature $|\psi_R\rangle$. However, ignoring $|\psi_R\rangle$ completely, σ is already irrefutable proof that the signer signed m . (We note that this simply shows that their construction cannot satisfy our stronger notion and does not constitute an attack on the MPY construction as per their definition.)

2.1 Definitions: Simulation-Style

Since it is possible for constructions to satisfy prior definitions, but still admit evidence-collection attacks, we need new definitions. In this section, we provide an in-depth discussion of our definitions and their merits. We use a simulation-style definition, which offers a robustness even against classes of attacks that have not yet been explicitly considered. As demonstrated by the evidence-collection on MPY24’s revocable signatures, such robustness is invaluable for avoiding subtleties that creative attackers might take advantage of.

Signatures. In certified deniability for signatures, we consider the following scenario that might happen in the real world:

1. The adversarial verifier V^* receives a public verification key vk .
2. V^* receives some signatures (possibly including on the same message multiple times) on some multi-set of message M over a period of time.
3. Over time, V^* outputs a list of certificates for some set of signatures. The signer decides whether to accept or reject each one.

To transform this scenario into a security experiment $\text{Sig-CDen-Exp}_{V^*}(\text{sk}, \text{vk})$, we consider outputting V^* ’s view along with some additional information. Let D be the set of messages associated with the signatures which the signer received valid certificates for. The experiment also outputs the multi-set of messages $M \setminus D$ which V^* for which received signatures but did not present a valid certificate.

Certified deniability requires that there exists a simulator $\mathcal{S}$ which receives vk and access to whichever signatures it wants, then produces a view that is indistinguishable from $\text{Sig-CDen-Exp}_{V^*}(\text{sk}, \text{vk})$ even when the list of messages $\mathcal{S}$ queries for is included in the output:

$$\{(\text{vk}, M \setminus D, \text{Sig-CDen-Exp}(\text{sk}, \text{vk}))\}_{(\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda)} \approx \left\{(\text{vk}, M_{\mathcal{S}}, \mathcal{S}^{\text{Sign}(\text{sk}, \cdot)}(\text{vk}))\right\}_{(\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda)}$$

Since the real adversary gets “free signatures” on messages in D , but all of $\mathcal{S}$ ’s queries are recorded, $\mathcal{S}$ must be able to produce its view *without* receiving the additional signatures in D .

A useful property of this definition is that it hides the number of messages which have been signed and subsequently deleted. To illustrate this, consider a scenario where V^* asks for a single signature on m^* , then deletes it honestly; the corresponding simulator receives no signatures. Intuitively, to be indistinguishable, the simulator must somehow convince the adversary that it has seen a valid signature, without actually forging one. A more relaxed definition might make this task easier by giving the simulator a signature $\sigma_\perp$ on $\perp$. In this case, the simulator must only convince the adversary that $\sigma_\perp$ is in fact a signature on m^* . This corresponds to revealing that *some* message was signed, if not which one. However, because our definition does not give *any* signatures to the simulator, it requires that after deletion, a third party cannot even tell whether a single signature was given out.

NIZKs. Certified deniability for NIZKs is defined similarly. In the real execution NIZK-CDen-Exp, the prover uses a witness w to prove the truth of some statement x . The simulator must reproduce the (adversarial) verifier’s view given only x , without a witness w .⁴ The reader may notice that this definition seems almost identical to the standard zero knowledge definition. Indeed, Pass [Pas03] points out that zero knowledge is inherently deniable in the plain model; it is indistinguishable whether the transcript given to you was the result of an honest execution, or the result of running the simulator. In the case where the statement is hard to decide, a simulator could even “prove” a false statement. Of course, NIZKs do not exist in the plain model [GO94]. Instead, we must consider the deniability of NIZKs in the random oracle model or the common reference string model.

Certified Deniability in the QROM. Traditionally for NIZKs in the random oracle model, the simulator is allowed to choose the random oracle, or similarly it is allowed to choose common reference string in the CRS model. However, Pass points out that in the real world, the random oracle is fixed once and for all. It is not realistic to believe that an arbitrary prover backdoored the random oracle. So, they could not have run the simulator.

Following Pass, we define certified deniability in the QROM so that the simulator does not have the ability to choose the random oracle. Specifically, in the ideal world, the simulator has access to a truly random oracle H , which also appears in the experiment output:

$$\{(H, \text{NIZK-CDen}^H(x, w))\}_{H \leftarrow \mathcal{H}} \approx \{(H, \mathcal{S}^H(x))\}_{H \leftarrow \mathcal{H}}$$

where $\mathcal{H}$ is the set of all functions $\{0, 1\}^{p(\lambda)} \rightarrow \{0, 1\}^{q(\lambda)}$.

The inclusion of the original oracle H prevents a simulator from reprogramming the random oracle. For example, if it were to reprogram the oracle to $H'(y) = v'$, instead of $H(y) = v$, then a distinguisher who queries the real H on y would immediately catch it. Without programmability, the task of simulating a NIZK becomes much harder. In fact, Pass utilizes the non-programmability enforced by this definition to show that (in the classical setting) deniable NIZKs are in fact *impossible* even with a random oracle.

The Power of Forgetting. We show that it is possible to avoid this issue in the quantum setting. Quantum mechanics offers the intriguing capability to force the adversary to *forget* information it queried on when it produces a valid deletion certificate. This enables the simulator to internally pretend that $H(y) = v'$ without later being detected by the distinguisher querying the real H on y , since the distinguisher never learns y .

We remark that in real world, where H is heuristically implemented, the third party does not actually believe that the implementation changed to be $H(y) = v$. However, we can heuristically say that if they do not know y , then they do not know anything about $H(y)$, so they have no evidence against $H(y) = v$ in the actual implementation either.

Relation to Other Primitives with Certified Deletion. Our new simulation-based certified deletion approach is stronger than some existing revocability definitions, but equivalent to others. For example, in revocable encryption, the simulator could simply encrypt 0 instead of m , since

⁴We remark that this only considers a single NIZK, in contrast with the signature definition which considered multiple signatures at a time. The difference between these is due to the need to sample a vk at the start of the signature experiment. Since NIZKs do not have secret information like this, they have better composeability properties.

the storage provider anyway does not have the decryption key before deletion time. However, the existing revocability definition for revocable signatures is weaker than our simulation definition.

Broadly, we can categorize revocable primitives as “passive” or “active”. Passive primitives, such as encryption or commitments, do now allow a user who holds the revocable object to utilize it even before revocation. Because they do not reveal information before deletion, existing definitions are generally equivalent to their simulation-based counterparts. On the other hand, active primitives, such as signatures [MPY24], arguments [HMNY22], or obfuscated programs [BGK⁺24], allow the user who holds a revocable object to utilize it until revocation. For these primitives, extra care must be taken to control *exactly* what information is retained after revocation. The simulation-based approach ensures that *only* the information which the user is intended to receive (e.g. a single bit indicating whether the signature was valid) is retained after revocation.

Before-the-Fact Coercion. [CGV22] previously introduced the idea of “before-the-fact” coercion for deniable encryption. This notion considers a coercer who approaches a victim before the victim computes a ciphertext (e.g. to cast a vote), then forces them to encrypt a particular message by requiring them to produce evidence that they encrypted that message. Even with this modification, the coercer should not be able to identify whether the encryptor encrypted a desired message m , or something else. Coledangelo, Goldwasser, and Vazirani showed that this notion is classical impossible, but possible to achieve using quantum techniques.

Signatures and NIZKs with certified deniability are naturally deniable even against before-the-fact coercion of the verifier. Even if the victimized verifier is forced to use a special device or obfuscated program to collect evidence about the received signature/NIZK, this evidence is rendered useless after the verifier produces a valid certificate. Classically, this notion is not possible even in the random oracle model, since the auxiliary program could record any ROM input-output behavior when run by the victim, enabling the coercer to check its correctness later. However, in the quantum setting, any such record might prevent the verifier from outputting a valid certificate since the certificate generation requires “forgetting” QROM queries. Furthermore, if the victim is forced to collect evidence anyway by not giving a valid certificate, then the authority can at least identify that they have been subject to coercion!

Does No-Query [MPY24] Satisfy the Simulation Definition? The reader might observe that the evidence-collection attack described earlier only applied to their multi-query-secure construction. For technical reasons related to a parallel repetition amplification step (see the next section), their no-query construction also allows evidence-collection.

2.2 Constructions

We now give an overview of our constructions, starting with signatures. For signatures, we will consider a selective notion of security where the adversary V^* must specify whether it later intends to delete each signature when it queries for that signature.

Starting Point. We start by recalling how [MPY24] constructs revocable signatures for the message “0” (i.e. 2-tier tokenized signatures). The signer samples a random pair of values (x_0, x_1) and a random phase $c \leftarrow \{0, 1\}$. Then, they compute one-way function images $f(x_0)$ and $f(x_1)$ and output

$$\text{vk} := (f(x_0), f(x_1)), \quad \text{sk} := (x_0, x_1, c), \quad |\psi_{\text{sig}}\rangle := |x_0\rangle + (-1)^c |x_1\rangle$$

Given this, a verifier can measure $|\psi\rangle$ and check the result matches either $f(x_0)$ or $f(x_1)$. This operation can be done coherently to avoid disturbing the state, enabling the verifier to delete after it verifies. The deletion certificate is obtained by measuring $|\psi\rangle$ in the Hadamard basis, which results in a vector d such that $c = d \cdot (x_0 \oplus x_1)$. Using the secret key, the signer can verify this certificate.

[MPY24] shows that the adversary cannot produce both such a d and one of x_0 or x_1 , except with probability $1/2$. Then, they show that repeating the scheme in parallel amplifies the difficulty of this task to a $\text{negl}(\lambda)$ success rate; no adversary can obtain both a Hadamard basis measurement of every index and a string containing one element from every pair (x_0, x_1) .

Attempt at a Fix. As discussed previously, a weakness in MPY24’s multi-query construction is that it directly signs the message m under the global verification key. We can avoid this issue by directly associating the local verification key with m using $H(x\|m)$ instead of $f(x)$ in the key vk_m . This is binding on m , but if the adversary “forgets” x after deletion, then $H(x\|m)$ looks uniformly random and independent of m . Now we can safely sign vk_m using the master signing key to validate fresh signatures $|\psi_{\text{vk}_m}\rangle$.

Of course, the underlying scheme only ensures $1/2$ “forgetfulness”. We cannot amplify security using parallel repetition because even a single retained instance, which can be cheated with probability $1/2$, is still irrefutable evidence. Instead, we secret share m using the additive λ -of- λ threshold secret sharing scheme, obtaining $m_1, \dots, m_\lambda$ such that $m_1 \oplus \dots \oplus m_\lambda = m$, and sign each share m_i using the modified scheme:

$$\text{Sign}(\text{sk}, H(x_0^i\|m_i)\|H(x_1^i\|m_i)), m_i, |\psi_i\rangle := |x_0^i\rangle + (-1)^{c_i}|x_1^i\rangle$$

If the adversary forgets even a single pair (x_0^i, x_1^i) , they would be unable to verify m_i . However, changing any m_i changes the signed message m .

Proof Technique: Forgetful Local Programming. Although this construction turns out to not fully satisfy certified deniability, it will be helpful in demonstrating our new technique: forgetful local programming. The simulator will require a signature σ_0 on any message, say on “0” (note that giving this to the simulator leaks the number of messages signed, if not which ones). It will attempt to locally convince the verifier that this signature is actually for m .

To do so, it picks a random index i and computes $m'_i = m_i \oplus m$. If m'_i were swapped for m_i , then the secret-shared message becomes $(m_1 \oplus \dots \oplus m_\lambda) \oplus m = 0 \oplus m = m$. To create this change, the simulator creates a modified oracle H' , which swaps the behavior of $x_b^i\|m_i$ with $x_b^i\|m'_i$, i.e.

$$\begin{aligned} H'(x_b^i\|m'_i) &:= H(x_b^i\|m_i) \\ H'(x_b^i\|m_i) &:= H(x_b^i\|m'_i) \end{aligned}$$

Then, it runs the adversarial verifier V^* using H' and σ_0 . If H' were the true oracle, then σ_0 is actually a signature on m .

The crux of the argument comes down to showing that any third party distinguisher cannot distinguish whether it has access to H or H' . Since σ_0 is a signature on 0 under H , but is a signature on m under H' , it therefore cannot tell which message σ_0 was originally associated with. To argue the crux, we will show that if V^* produces a valid certificate, then the distinguisher cannot find either x_0^i or x_1^i .

The Need for a New Approach. Unfortunately, this approach only achieves $1/\text{poly}(\lambda)$ security.⁵ The issue is that the adversary could simply guess i , and is correct with inverse polynomial probability. If it deletes the other indices honestly, then it can keep one of x_0^i and x_1^i with probability $1/2$ even while providing a valid deletion certificate. Then, given x_b^i , the distinguisher could immediately detect the local reprogramming. Without the forgetful local programming technique, the tools of constructing certifiably deniable NIZKs becomes very close to what is available in the classical setting, where we know the task is impossible.

One might hope that by increasing the number of values in superposition, e.g. by constructing $\sum_i (-1)^{c_i} |x_i\rangle$ instead of $|x_0\rangle \pm |x_1\rangle$, the likelihood that the adversary can keep an x_i while simultaneously producing a valid deletion certificate can be decreased. While this is true, a superposition over many x_i requires a proportionate number of signed random oracle images $H(x_i||m)$. Increasing the “forgetfulness” of the adversary to overwhelming probability would blow up the size of the signature super-polynomially.

Another issue is that this approach allows the adversary to collective evidence that *at least one* message was signed, if not which message it was. In other words, the approach does not ensure that the *number* of signatures is deleted. This appears in the security sketch as the requirement that the simulator receives a post-quantum signature σ_0 , which it cannot forge on its own.

Idea 1: Subspace States. To increase the “forgetfulness” of the adversary beyond polynomial factors, we turn to subspace states [AC12]. A subspace state $|A\rangle \propto \sum_{a \in A} |a\rangle$ is a uniform superposition over elements of an $\lambda/2$ -dimensional subspace A of $\mathbb{F}_2^\lambda$. Using them, we can avoid the exponential blow-up by signing the random oracle images in superposition:

$$|\sigma_m\rangle := \sum_{a \in A \setminus \{0\}} |a\rangle \otimes |\text{Sign}(\text{sk}, H(a||m))\rangle$$

The verifier can check such a state by (1) coherently running the signature verification procedure in the computational basis and (2) coherently checking that the signed message matches $H(a||m)$. To delete the signature, they can simply return the whole state. If the signing algorithm is deterministic, then the signer can check the certificate by uncomputing the signature and random oracle image, then checking that the certificate is now $|A\rangle$ using a projection onto $|A\rangle$. We note that it is possible to make the signing algorithm deterministic by coherently deriving its randomness from a PRF evaluated at a , e.g. $H(k||a)$ for random k .

We argue that this deletion check enforces “forgetfulness”. Due to the direct product hardness property [BS23], we know that given a random subspace state $|A\rangle$, it is hard to find both a vector in $A \setminus \{0\}$ and a vector in $A^\perp \setminus \{0\}$, even given access to an oracle which decides membership in A and $A^\perp$. The membership oracle can be used to check if the returned certificate indeed contains an intact copy of $|A\rangle$. If the signer is able to recover $|A\rangle$ from the certificate, then it could obtain a vector in $A^\perp$ by measuring it in the Hadamard basis. Whenever this happens, the verifier cannot also remember *any* vector in A , other than 0. Thus, we can use the forgetful local programming technique to obtain $\text{negl}(\lambda)$ security loss.

Idea 2: Fiat-Shamir in Superposition. This still leaves the issue of leaking the number of signatures. To solve this, we use the Fiat-Shamir paradigm in superposition. Fiat-Shamir

⁵Actually, this cannot be black-box reduced to the hardness of finding x_0^i or x_1^i discussed above, because of a large loss that occurs when extracting QROM queries. This problem can be fixed by outputting the whole signature as the certificate and doing some additional technical work, though we omit the details since it is subsumed by our later construction.

transforms a sigma protocol⁶ into a signature scheme. The signer samples a random secret key $\mathbf{sk}$ and gives out $\mathbf{vk} = f(\mathbf{sk})$ as the verification key, where f is a one-way function. To sign a message m , the signer computes a sigma protocol proving knowledge of an $\mathbf{sk}$ matching $\mathbf{vk}$. Fiat-Shamir uses the first message s_1 of the sigma protocol to derive the second message s_2 as $H(m\|\mathbf{vk}\|s_1)$.

Performing the Fiat-Shamir signature in superposition yields:

$$|\sigma_m\rangle := \sum_{a \in A \setminus \{0\}} |a\rangle \otimes |s_1^a, s_2^a = H(a\|m\|\mathbf{vk}\|s_1), s_3^a\rangle$$

where (s_1^a, s_3^a) are the prover's messages in the sigma protocol using randomness $H(k\|a)$. Verification and deletion are defined similarly to the signature construction above.

Also similarly to the previous construction, if the certificate is valid, then the verifier must have “forgotten” every element of $A \setminus \{0\}$. Thus, even if it knew some transcript (s_1^a, s_2^a, s_3^a) of the sigma protocol, it could not prove to a third party that s_2^a was really derived using $H(a\|m\|\mathbf{vk}\|s_1)$.⁷ Instead, the third party would suspect that s_2^a was chosen carefully to match a faulty s_1 .

Crucially, by locally programming H at points that include some $a \in A$, the simulator can simulate every Fiat-Shamir transcript (s_1^a, s_2^a, s_3^a) *only using knowledge of* $\mathbf{vk}$ (or of the statement x). Thus, in the signature case, it no longer needs to receive anything signed under $\mathbf{vk}$ to do its job.

NIZKs. A useful consequence of using Fiat-Shamir to construct signatures is that the Fiat-Shamir transform can also be used for turning a sigma protocol into a NIZK. The construction is similar to the signature case, except that $s_2^a = H(a\|m\|\mathbf{vk}\|s_1^a)$ is replaced with $H(a\|x\|s_1^a)$, where x is the NP statement being proven.

Other Technical Challenges. We briefly mention two additional technical challenges that appear in our construction. First, it is not immediately obvious that Fiat-Shamir can be simulated in superposition, even if it is post-quantum secure. Previous works have addressed such issues by using collapsing protocols [Unr16b] or small-range distributions [Zha12]. However, both of these techniques require collapsing the argument/signature to a large degree, which is at odds with deletion: if a superposition state is indistinguishable from a measured state, we are almost back to the classical case and there is no way to delete! To avoid this issue, we use complexity leveraging to switch each of the superposed transcripts to be simulated, one at a time.

Second, it is not immediately obvious that Fiat-Shamir is sound in a structured superposition as above. The soundness of Fiat-Shamir in the quantum setting is a highly nontrivial task, but has been shown under certain conditions in the case where the resulting argument is classical [DFMS19, LZ19]. To argue soundness of Fiat-Shamir in superposition, we actually use coset states. We show that if the coset offset is not known, then the coset state appears to have been measured in the computational basis. Thus, we can treat our construction as having a classical argument/signature when proving soundness.

2.3 Black-Box Barriers to Plain Model Constructions.

Finally, we give an overview of the black-box barrier for the plain model. We model the simulator as having black-box access to the unitary dilation of the adversary and its inverse, but it may not

⁶A sigma protocol is a 3-message public-coin argument of knowledge which is zero-knowledge against an honest verifier whose second message is known ahead of time.

⁷It is tempting to try to use a single first message s_1 and derive the second and third messages of the sigma protocol in superposition using a . However, this would lead to answering multiple challenges using the same first message, which may not be possible with a simulated s_1 .

directly access the adversary’s internal registers, e.g. its auxiliary input.

Consider an adversary Adv and a distinguisher D which, as part of their auxiliary input, share a program that includes a key pair $(\tilde{\text{sk}}, \tilde{\text{vk}})$ for an internal (post-quantum) signature scheme. The program on some input an alleged signature $|\sigma\rangle$ for a message m under key vk , verifies it and if the check passes, it signs $m\|\text{vk}$ using the internal signature key $\tilde{\text{sk}}$. This operation to create a proof that m was signed is gentle because of the correctness of the candidate scheme, so Adv can still generate a valid certificate using the honest deletion algorithm, after it has obtained a proof for the distinguisher that m was signed. Now it has produced both a valid certificate and a post-quantum signature on $m\|\text{vk}$. The distinguisher can simply check the latter using $\tilde{\text{vk}}$.

Observe that in the real world, D will almost always obtain a valid signature on $m\|\text{vk}$. On the other hand, the simulator cannot hope to extract such a signature using black-box access to Adv , unless it is able to forge signatures for the candidate scheme.

On a technical level, the analysis requires generalizing a technique introduced by [BBBV97] for analyzing the behavior of an oracle algorithm with a reprogrammed *classical* oracle to handle oracles that do quantum computation instead. Roughly, we show that if an oracle algorithm is able to distinguish between oracle access to two unitaries U_0 and U_1 , then outputting its query register at a random timestep produces a mixed state with noticeable probability mass on pure states $|\psi\rangle$ where $U_0|\psi\rangle$ and $U_1|\psi\rangle$ are proportionally far in trace distance. The generalized technique may be of independent interest.

Relation to Program Obfuscation. The above sketch considers a model where the simulator has black-box access to the unitary dilation of the adversary, but *cannot* access the adversary’s auxiliary input, except indirectly by querying the adversary. We could also consider a model where the simulator *can* access the auxiliary input. In this case, if the auxiliary input consists of a ideally-obfuscated program which has the signing key hard-coded, then the simulator’s access to the signing key becomes identical to the model where the auxiliary input cannot be directly accessed. The fact that ideal obfuscation causes the two models to become identical suggests that any plain model construction would need to use non-black-box techniques to bypass potentially obfuscated auxiliary input.

Avoiding the Barrier with QROM. The quantum random oracle bypasses the black-box issue for a few reasons. First, reprogramming a random oracle is, in a sense, a non-black-box operation. Second, the black-box issue arises because the simulator needs to somehow trick the adversary into thinking it has received a signature on m^* , without actually receiving a signature. However, in the plain model, the simulator does not seem to have more power than a real adversary, who should not be able to forge signatures. Introducing the ability to temporarily reprogram the oracle via the forgetful reprogramming technique restores the necessary asymmetry between the simulator and a real adversary.

2.4 Related Works

Concurrent Work. [AK24] concurrently proposed NIZKs with certified deletion. Their definition ensures that no man-in-the-middle can receive a NIZK and simultaneously produce a valid certificate along with a NIZK that is accepting with respect to the *honest verification procedure* unless they already know a witness. This definition has similarities to [MPY24]’s revocable signature definition in that it only considers third party verifiers who run the honest verification procedure. Their construction does not satisfy certified deniability, in part due to the use of a parallel amplifi-

cation step (see the technical overview). They do achieve classical certificates in their construction, which we leave open for certified deniability.

Quantum Deniability. [CGV22] revisit the problem of deniable encryption in the quantum setting. In classical deniability, the encryptor should be able to produce “fake” proof to the adversarial coercer how a given ciphertext is actually an encryption of some other message. Coledangelo, Goldwasser, and Vazirani propose a uniquely quantum spin on the task: by computing the ciphertext, any explanation for it is destroyed. Although this has similarities to our setting, their result is quite different. They ensure that the third-party coercer never sees the explanation. In contrast, the adversarial verifier necessarily sees the “explanation” for signatures/NIZKs - the signature/NIZK itself - but later is forced to “forget” it.

Certified Deletion. Certified deletion was first proposed by Broadbent and Islam for encryption [BI20]. It has since been generalized to a variety of other primitives, e.g. [HMNY22, HKM⁺24, Por23, APV23, AKN⁺23, BK23, BGK⁺24, BR24, MPY24]. To the best of our knowledge, the only two works to have considered any notion of simulation in defining certified deletion are [HMNY22] and [BGK⁺24]. [HMNY22] considers certified everlasting zero knowledge, which uses a simulation definition as a result of standard definitions for zero knowledge. [BGK⁺24] consider a simulation-style definition of obfuscation with certified deletion in the structured oracle model as a side result, inspired by definitions of ideal obfuscation. In contrast, signatures are not typically considered to be a “simulation primitive”, and NIZKs require either the random oracle or CRS models, which require special care for deniability.

Unclonability. Unclonability prevents an adversary from transforming an object (such as a program) into two functioning copies of the object [Aar09]. It is closely related to certified deletion, since a functional copy of the object can be considered as the “certificate”. Previously, Goyal, Malavolta, and Raizes [GMR24] considered a related notion to certified deniability under the name of “strongly unclonable proofs”. In a strongly unclonable proof, an adversarial man-in-the-middle (MiM) who receives a simulated proof of some (potentially false) statement x , then interacts with two sound verifiers to prove statements $\tilde{x}_1$ and $\tilde{x}_2$. Strong unclonability guarantees that no MiM can convince *both* verifiers of false statements. GMR showed that in general, strongly unclonable NIZKs do not exist. Fortunately, their techniques do not extend to certified deniability. GMR’s impossibility relies on an interactive verification, during which the MiM can forward messages between the two verifiers. In certified deniability, the NIZK is deleted before any messages reach the second verifier. In general, our definition is also more robust; for example, it rules out the possibility of the third-party verifier who accepts false statements with probability $1/2$ from being convinced by a deleted NIZK with even probability $1/2 + \epsilon$.

3 Preliminaries

All assumptions mentioned are post-quantum unless otherwise specified. We write $\text{Func}(\mathcal{X}, \mathcal{Y})$ to be the set of all functions $f : \mathcal{X} \rightarrow \mathcal{Y}$. We say that two distributions A and B are computationally indistinguishable, denoted as $A \approx_c B$, if no QPT distinguisher D with poly-size auxiliary input register R_D can distinguish between them with noticeable advantage. When we wish for a finer-grained treatment, we say that A and B are ϵ -computationally indistinguishable, denoted as $A \approx_c^\epsilon B$, if the distinguishing advantage is instead bounded by ϵ . In the case where A or B depends on a poly-sized auxiliary input register R_A , the contents of R_A and R_D may be entangled.

3.1 Quantum Computing

Subspace States. For any subspace $A \subset \{0, 1\}^\lambda$, the subspace state $|A\rangle$ for A is

$$|A\rangle : \propto \sum_{a \in A} |a\rangle$$

Aaronson and Christiano [AC12] show that the projector onto $|A\rangle$ can be implemented using queries to membership oracles O_A and $O_{A^\perp}$ that decide the query's membership in A (respectively, $A^\perp$).

Lemma 1 ([AC12]). *Let $A \subset \{0, 1\}^\lambda$ be a subspace. Let $P_A = \sum_{a \in A} |a\rangle\langle a|$ and $P_{A^\perp} = \sum_{a \in A^\perp} |a\rangle\langle a|$ be projectors onto the space spanned by elements of A and $A^\perp$, respectively. Then*

$$H^{\otimes n} P_{A^\perp} H^{\otimes n} P_A = |A\rangle\langle A|$$

Ben-David and Sattath show that given a random subspace state of appropriate dimension, it is hard to find a vector $v_1 \in A \setminus \{0\}$ together with a vector $v_2 \in A^\perp \setminus \{0\}$, even when given oracle access O_A and $O_{A^\perp}$.

Lemma 2 ([BS23]). *Let $A \subset \{0, 1\}^\lambda$ be a random subspace of dimension $\lambda/2$ and let $\epsilon > 0$ be such that $1/\epsilon = o(2^{\lambda/2})$. Given one copy of $|A\rangle$ and oracle access to O_A and $O_{A^\perp}$, any adversary who produces $v_1 \in A \setminus \{0\}$ together with $v_2 \in A^\perp \setminus \{0\}$ with probability ϵ requires $\Omega(\sqrt{\epsilon} 2^{\lambda/2})$ queries.*

Quantum Oracles. We recall a result from [BBBV97] that aids in reasoning about reprogrammed oracles. Consider a quantum adversary who has quantum query access to one of two classical oracles H and H' . They bound the ability of the adversary to distinguish between the two oracles in terms of the amplitude with which it queries on (classical) inputs x where $H(x) \neq H'(x)$. As a simple corollary, if the adversary is able to distinguish the two oracles in a polynomial number of queries, then measuring one of its queries at random produces an x such that $H(x) \neq H'(x)$ with noticeable probability.

Lemma 3 ([BBBV97], Paraphrased). *Let H and H' be oracles which differ on some set of inputs $\mathcal{X}$. Let $|\psi_i\rangle = \sum_y \alpha_{y,i} |\phi_{y,i}\rangle \otimes |y\rangle_Q$ be the state of Adv^H at time i , where Q is the query register. Let $|\psi'_i\rangle$ be the state of $A^{H'}$ at time i . Then for all $T \in \mathbb{N}$,*

$$\text{TD}(|\psi_T\rangle, |\psi'_T\rangle) \leq \sqrt{T \sum_{i=1}^T \sum_{x^* \in \mathcal{X}} |\alpha_{x^*,i}|^2}$$

where TD denotes the trace distance.

Quantum Random Oracles. In the quantum random oracle model, all parties have access to an oracle $H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})$ implementing a random function. To ease notation, we implicitly pad inputs to the random oracle: given $x \in \mathcal{X}_1$, where $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$, we denote $H(x \| \vec{0})$ as $H(x)$.

It will be useful to sometimes be able to derive large amounts of randomness from the oracle as if it were a PRF. The following lemma formalizes this treatment.

Lemma 4. *Let $H : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{Y}$ and $G : \mathcal{X}_2 \rightarrow \mathcal{Y}$ be random oracles. Define $H_k : \mathcal{X}_2 \rightarrow \mathcal{Y}$ by $H_k(v) := H(k \| v)$. If $1/|\mathcal{X}_1| = \text{negl}(\lambda)$, then*

$$\left\{ (O_H, O_{H_k}) : \begin{array}{c} H \leftarrow \text{Func}(\mathcal{X}_1 \times \mathcal{X}_2, \mathcal{Y}) \\ k \leftarrow \mathcal{X}_1 \end{array} \right\} \approx_c \left\{ (O_H, O_G) : \begin{array}{c} H \leftarrow \text{Func}(\mathcal{X}_1 \times \mathcal{X}_2, \mathcal{Y}) \\ G \leftarrow \text{Func}(\mathcal{X}_2, \mathcal{Y}) \end{array} \right\}$$

where O_f denotes oracle access to a function f .

Proof. For any $k \in \mathcal{X}_1$, define $H_{k,G}$ as

$$H_{k,G}(x) = \begin{cases} G(x') & \text{if } x = k\|x' \text{ for some } x' \\ H(x) & \text{else} \end{cases}$$

The right distribution is identically distributed to

$$\left\{ (O_{H_{k,G}}, O_{H_k}) : \begin{array}{l} H \leftarrow \text{Func}(\mathcal{X}_1 \times \mathcal{X}_2, \mathcal{Y}) \\ G \leftarrow \text{Func}(\mathcal{X}_2, \mathcal{Y}) \end{array} \right\}$$

By [Lemma 3](#), any distinguisher who distinguishes $(O_{H_{k,G}}, O_{H_k})$ from (O_H, O_{H_k}) with advantage ϵ in q queries can produce some x^* such that $O_{H_{k,G}}(x^*) \neq O_H(x^*)$ with probability ϵ^2/q^2 . Whenever this occurs, $x^* = k\|x$ for some x . Since k is drawn uniformly at random from $\mathcal{X}_1$, it must be the case that $\epsilon^2/q^2 \leq 1/|\mathcal{X}_1| = \text{negl}(\lambda)$. If q is $\text{poly}(\lambda)$, then ϵ must be $\text{negl}(\lambda)$. $\square$

Z-Twirl. It is well-known that adding a random phase to a state is equivalent to measuring the state in the computational basis. Here we present a slightly generalized form of this.

Lemma 5. *Let $|\psi\rangle = \sum_{x \in \mathcal{X}} \alpha_x |x\rangle \otimes |\phi_x\rangle$ be a quantum state where $\mathcal{X}$ is a vector space. Denote $|\psi_s\rangle := \sum_{x \in \mathcal{X}} \alpha_x (-1)^{s \cdot x} |x\rangle \otimes |\phi_x\rangle$ for any $s \in \mathcal{X}$. Then*

$$\frac{1}{|\mathcal{X}|} \sum_{s \in \mathcal{X}} |\psi_s\rangle \langle \psi_s| = \sum_{x \in \mathcal{X}} |\alpha_x|^2 |x\rangle \langle x| \otimes |\phi_x\rangle \langle \phi_x|$$

Proof. We compute

$$\begin{aligned} \frac{1}{|\mathcal{X}|} \sum_{s \in \mathcal{X}^n} |\psi_s\rangle \langle \psi_s| &= \frac{1}{|\mathcal{X}|} \sum_{s \in \mathcal{X}} \alpha_{x_1} \overline{\alpha_{x_2}} \sum_{x_1, x_2 \in \mathcal{X}} (-1)^{(x_1 - x_2) \cdot s} |x_1\rangle \langle x_2| \otimes |\phi_{x_1}\rangle \langle \phi_{x_2}| \\ &= \frac{1}{|\mathcal{X}|} \sum_{x_1, x_2 \in \mathcal{X}} \alpha_{x_1} \overline{\alpha_{x_2}} |x_1\rangle \langle x_2| \otimes |\phi_{x_1}\rangle \langle \phi_{x_2}| \sum_{s \in \mathcal{X}} (-1)^{(x_1 - x_2) \cdot s} \\ &= \sum_{x \in \mathcal{X}} |\alpha_x|^2 |x\rangle \langle x| \otimes |\phi_x\rangle \langle \phi_x| \end{aligned}$$

$\square$

3.2 Argument Systems

Definition 1 (Argument). *An argument system (P, V) for an NP language $\mathcal{L}$ is a (potentially interactive) protocol between a prover P and a verifier V where P inputs a statement and a witness and V outputs accept or reject. It should satisfy two properties:*

- **Correctness.** *If w is a witness for $x \in \mathcal{L}$, then at the end of the execution, V outputs accept.*
- **Soundness.** *For every adversarial prover P^* , if $x \notin \mathcal{L}$, then at the end of an execution $\langle P^*, V \rangle$, V outputs reject with probability $1 - \text{negl}$. If this holds for QPT P^* , we call the soundness computational. If it holds for unbounded P^* , we call the soundness statistical.*

If the argument system is non-interactive, we consider P and V to consist of single operations Prove and Verify.

Definition 2 (Zero-Knowledge). *An argument system (P, V) for an NP language $\mathcal{L}$ is zero knowledge if there exists a QPT algorithm $\mathcal{S}$ such that for all QPT adversaries V^* with auxiliary input register R_{V^*} and all statement/witness pairs (x, w) where $x \in \mathcal{L}$,*

$$\{\langle P(x, w), V^*(R_{V^*}) \rangle\} \approx_c \{\mathcal{S}(V^*, R_{V^*})\}$$

A NIZK is simply a zero-knowledge argument with only a single message (i.e. it is non-interactive).

Another useful property for argument systems to hold is to be a proof of knowledge. This ensures that the prover must “know” a witness for the statement it is proving. Since we do not directly use the proof of knowledge property in this work, we refer the reader to [Unr12, LZ19].

A frequently useful class of argument arises from sigma protocols. Sigma protocols are three round, public-coin arguments with relaxed zero-knowledge properties. In a public-coin protocol, the verifier’s messages are truly random.

Definition 3 (Sigma Protocol). *A Sigma protocol for an NP language $\mathcal{L}$ with relation $\text{Rel}_{\mathcal{L}}$ is a three-message public-coin argument system $\Sigma = (P_1, P_3, \text{Verify}_{\Sigma})$ for $\mathcal{L}$ with the following properties:*

- **Special Soundness.** *There exists an extractor E which, given two transcripts (s_1, s_2, s_3) and (s_1, s'_2, s'_3) for the same statement x with the same first message s_1 and different verifier challenges $s_2 \neq s'_2$, extracts a witness for x .*
- **Special Honest-Verifier Zero Knowledge (HVZK).** *There exists a QPT simulator $\mathcal{S}_{\Sigma}$ such that for every $(x, w) \in \text{Rel}_{\mathcal{L}}$ and every second message s_2 ,*

$$\{(s_1, s_2, s_3) : s_1 \leftarrow P_1(x, w), s_3 \leftarrow P_3(x, w, s_2)\} \approx_c \{\mathcal{S}_{\Sigma}(x, s_2)\}$$

We say Σ is ϵ -secure if no QPT distinguisher has greater than ϵ advantage in distinguishing these two distributions.

Fiat-Shamir Transform. The Fiat-Shamir transform [FS87, BG93] modifies a sigma protocol to become non-interactive by using a random oracle H . Specifically, the prover first computes $s_1 \leftarrow P_1(x, w)$, then computes the verifier’s challenge $s_2 \leftarrow H(x \| s_1)$, and finally computes $s_3 \leftarrow P_3(x, w, s_2)$ and outputs (s_1, s_2, s_3) . To verify a transcript, the verifier checks that $s_2 = H(s_1)$, then verifies the transcript using Verify_{Σ} . Given a sigma protocol Σ , we denote the Fiat-Shamir transform of Σ as $\text{FS}_{\Sigma}^H(x, w)$. We extend the notation as $\text{FS}_{\Sigma}^H(x, w; r)$ when specifying the prover’s randomness r for Σ .

Although the analysis of Fiat-Shamir is more complicated in the quantum setting, a series of works have shown that, under mild assumptions on the sigma protocol, post-quantum Fiat-Shamir is sound even with quantum access to H , culminating in [DFMS19, LZ19].

Theorem 3 ([LZ19]). *If a post-quantum sigma protocol has (1) perfect completeness, (2) quantum proof of knowledge, and (3) unpredictable first messages, then the Fiat-Shamir heuristic gives a quantum NIZKPoK.*

Their result requires that the sigma protocol has unpredictable first messages, which requires that the probability of two executions having the same first message is negligible for all (x, w) :

$$\Pr[s_1 = s'_1 : s_1 \leftarrow P_1(x, w), s_2 \leftarrow P_1(x, w)] = \text{negl}(\lambda)$$

Fiat-Shamir can also be used to create a signature scheme from a sigma protocol by using an NP instance x where finding the witness is hard (e.g. the image of a one-way function) as the public

key and its witness w as the secret key. A signature on a message m is obtained by computing $\text{FS}_\Sigma^{H(m\|\cdot)}(x, w)$, where $H(m\|\cdot)$ denotes H with the first portion of the input fixed to m .

[DFMS19, LZ19] also showed that if the underlying sigma protocol is collapsing,⁸ then the Fiat-Shamir transform gives a secure signature scheme in the quantum random oracle model.

Definition 4 (Collapsing for Sigma Protocols [Unr16a, DFMS19, LZ19, CMSZ22]). *We say a protocol $\langle P, V \rangle$ is collapsing if for every polynomial-size interactive quantum adversary P^* and polynomial-size quantum distinguisher Adv ,*

$$|\Pr[1 \leftarrow \text{CollapseExp}(0, P^*, \text{Adv})] - \Pr[1 \leftarrow \text{CollapseExp}(1, P^*, \text{Adv})]| \leq \text{negl}(\lambda)$$

For $b \in \{0, 1\}$, the experiment $\text{CollapseExp}(b, P^*, \text{Adv})$ is defined as follows:

1. The challenger executes $\langle P^*, V \rangle$, storing the result in registers $(R_1, \dots, R_n)$. It measures every register R_n in the computational basis.
2. The challenger coherently evaluates $V(|m_1, \dots, m_n\rangle)$ and measures the result. If it is reject, the experiment aborts by outputting a random bit.
3. If $b = 0$, the challenger does nothing. If $b = 1$, the challenger measures R_n in the computational basis.
4. The challenger sends $(R_1, \dots, R_n)$ to Adv . The experiment outputs Adv 's output bit.

Theorem 4 ([LZ19]). *If a post-quantum sigma protocol is collapsing, then the Fiat-Shamir heuristic gives a secure post-quantum digital signature scheme in the quantum random oracle model.*

We note that natural sigma protocols satisfying the requirements of both **Theorems 3** and **4** are known. For instance, Unruh [Unr12] shows that a slight modification of Blum's Hamiltonian path argument is a quantum proof of knowledge. It also has perfect completeness and unpredictable first messages. Furthermore, if the commitments in the first round are implemented via a random oracle, then it has subexponential HVZK.

3.3 Revocable Signatures and NIZKs

Definition 5. *A digital signature is a tuple of algorithms $(\text{Gen}, \text{Sign}, \text{Verify})$ with the following behavior:*

- $\text{Gen}(1^\lambda)$ takes in the security parameter and outputs a key pair (sk, vk) .
- $\text{Sign}(\text{sk}, m)$ takes in a signing key sk and a message m , then outputs a (potentially quantum) signature σ .
- $\text{Verify}(\text{vk}, \sigma, m)$ takes in a verification key vk , an alleged signature σ , and a message m , then outputs accept or reject.

A digital signature must satisfy the following properties:

- **Correctness.** For every message m :

$$\Pr \left[\text{Accept} \leftarrow \text{Verify}(\text{vk}, \sigma, m) : \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \\ \sigma \leftarrow \text{Sign}(\text{sk}, m) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

⁸[DFMS19] refers to this property as “computational unique responses”.

- **Existential Unforgeability under Chosen Message Attack (EUF-CMA).** For all QPT adversaries Adv ,

$$\Pr \left[\begin{array}{c} m \notin M \\ \wedge \\ \text{Accept} \leftarrow \text{Verify}(\text{vk}, \text{R}_{\text{sig}}, m) \end{array} : \begin{array}{c} (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \\ \text{Adv}^{\text{Sign}(\text{sk}, \cdot)}(\text{vk}) \end{array} \right] \leq \text{negl}(\lambda)$$

where M is the list of messages that the adversary queries to the signing oracle $\text{Sign}(\text{sk}, \cdot)$.

Revocable Signatures. A revocable signature scheme [MPY24] augments the signature scheme syntax with two additional algorithms Del and DelVer . Additionally, $\text{Sign}(\text{sk}, m)$ is modified to output both a signature register R_{sig} and a deletion verification key dk . The new algorithms act as follows:

- $\text{Del}(\text{R}_{\text{sig}})$ takes in a register containing a signature, then outputs a certificate register R_{cert} .
- $\text{DelVer}(\text{dk}, \text{R}_{\text{cert}})$ takes in the deletion verification key and a certificate register, then outputs accept or reject.

Additionally, a revocable signature scheme should satisfy deletion correctness and a notion of revocation security. We omit [MPY24]’s notion of revocation security here. Instead, we define certified deniability for (revocable) signatures in Section 4.1.

Definition 6 (Deletion Correctness). For all messages m ,

$$\Pr \left[\begin{array}{c} \text{Accept} \leftarrow \text{DelVer}(\text{dk}, \text{R}_{\text{cert}}) \\ (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \\ (\text{R}_{\text{sig}}, \text{dk}) \leftarrow \text{Sign}(\text{sk}, m) \\ \text{R}_{\text{cert}} \leftarrow \text{Del}(\text{R}_{\text{sig}}) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

Revocable NIZKs. Although these have not be explicitly defined before, they follow similar syntax to revocable signatures, so we include the description in this section. A revocable NIZK is augmented with two additional algorithms Del and DelVer , which act similarly to their signature counterparts. Additionally, a revocable NIZK must satisfy deletion correctness. We define certified deniability for (revocable) NIZKS in Section 4.2.

4 Definitions of Certified Deniability

4.1 Signatures

Deniable authentication was initially defined by Dwork, Naor, and Sahai [DNS98] using the simulation paradigm. Informally, a signature is deniable if it could be *simulated* by using only public information.

We follow a similar simulation-based paradigm which uses a real experiment $\text{Sig-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}(\text{vk})$. This experiment is parameterized by a QPT adversary Adv who receives auxiliary input in register R_{Adv} and a key pair (sk, vk) . It is defined as follows.

1. Adv is initialized with register R_{Adv} and vk . The challenger is initialized with sk .
2. Adv gets access to a signing oracle for sk . Each time the signing oracle is queried for some m_i (including duplicates), it adds m_i to a multi-set M and appends the deletion verification key dk_i for the signature it gives out to an internal log.

3. The adversary outputs a list of certificate registers $\{R_{\text{cert},i}\}_i$ and register R_{out} .
4. For each i where $\text{DelVer}(dk_i, R_{\text{cert},i}) = \text{Accept}$, add m_i to a multi-set D .
5. Output $(M \setminus D, R_{\text{out}})$.

If working in an oracle model where the parties have access to an oracle H , then we denote the experiment as $\text{Sig-CDen}_{\text{Adv}(R_{\text{Adv}})}^H$.

Definition 7 (Certified Deniability for Signatures: Plain Model). *A revocable signature scheme (see Section 3.3) $(\text{Gen}, \text{Sign}, \text{Verify}, \text{Del}, \text{DelVer})$ is certifiably deniable if for every QPT adversary Adv , there exists a QPT simulator $\mathcal{S}$ such that for every QPT adversary Adv with poly-size auxiliary input register R_{Adv} ,*

$$\begin{aligned} & \left\{ (\text{vk}, \text{Sig-CDen}_{\text{Adv}(R_{\text{Adv}})}(\text{sk}, \text{vk})) : (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \right\} \\ & \approx_c \\ & \left\{ (\text{vk}, M_{\mathcal{S}}, \mathcal{S}^{\text{Sign}(\text{sk}, \cdot)}(\text{Adv}, R_{\text{Adv}}, \text{vk})) : (\text{sk}, \text{vk}) \leftarrow \text{Gen}(1^\lambda) \right\} \end{aligned}$$

where $M_{\mathcal{S}}$ is the set of messages on which $\mathcal{S}$ queries $\text{Sign}(\text{sk}, \cdot)$. We say that the scheme has selective certified deniability if this condition holds for the following modified $\text{Sig-CDen}_{\text{Adv}(R_{\text{Adv}})}(\text{sk}, \text{vk})$ experiment. At the start of the experiment, after receiving vk , the adversary declares the set of messages it will delete. Additionally, each time it queries the signing oracle for m_i , it declares whether it intends to delete the signature for m_i . At the end of the experiment, if the multi-set D does not match the set of declared signatures, then the experiment outputs $(M \setminus D, \perp)$ instead of $(M \setminus D, R_{\text{Adv}})$.

The set M acts as a way to restrict $\mathcal{S}$ from querying on signatures which Adv decides to delete (potentially after seeing vk and other signatures). Adv gets to remove signatures from M in the experiment output by deleting them, but all of $\mathcal{S}$'s queries are recorded.

In the selective case, the adversary *only* has to declare the messages/signatures it intends to delete. It is free to receive other signatures, even adaptively. However, if it does not delete the signatures which it says it will delete, then the experiment aborts.

Certified Deniability in the QROM. We may also define certified deniability in the CRS model or the QROM model. Following [Pas03]'s definition from the classical setting, a deniable simulator does *not* have the ability to program the global random oracle; this is enforced by sampling a fresh random oracle before the experiment and including its description in the output of the experiment. Thus, any simulator which attempts to pretend that the oracle had different behavior is will be caught.

Definition 8 (Certified Deniability for Signatures: QROM). *A revocable signature scheme $(\text{Gen}, \text{Sign}, \text{Verify}, \text{Del}, \text{DelVer})$ is certifiably deniable in the quantum random oracle model if there exists a QPT simulator $\mathcal{S}$ such that for every QPT adversary Adv with poly-size auxiliary input register R_{Adv} ,*

$$\begin{aligned} & \left\{ (O_H, \text{vk}, \text{Sig-CDen}_{\text{Adv}(R_{\text{Adv}})}^H(\text{sk}, \text{vk})) : \begin{array}{l} H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \\ (\text{sk}, \text{vk}) \leftarrow \text{Gen}^H(1^\lambda) \end{array} \right\} \\ & \approx_c \\ & \left\{ (O_H, \text{vk}, M_{\mathcal{S}}, \mathcal{S}^{H, \text{Sign}^H(\text{sk}, \cdot)}(\text{Adv}, R_{\text{Adv}}, \text{vk})) : \begin{array}{l} H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \\ (\text{sk}, \text{vk}) \leftarrow \text{Gen}^H(1^\lambda) \end{array} \right\} \end{aligned}$$

where O_H denotes oracle access to H and M_S is the set of messages on which $\mathcal{S}$ queries $\text{Sign}^H(\text{sk}, \cdot)$. We say that the scheme has selective certified deniability in the QROM if this condition holds for the following modified $\text{Sig-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}^H(\text{sk}, \text{vk})$ experiment. At the start of the experiment, after receiving vk , the adversary declares the set of messages it will delete. Additionally, each time it queries the signing oracle for m_i , it declares whether it intends to delete the signature for m_i . At the end of the experiment, if the multi-set D does not match the set of declared signatures, then the experiment outputs $(M \setminus D, \perp)$ instead of $(M \setminus D, \text{R}_{\text{Adv}})$.

We show in [Section 5](#) how to obtain signatures with certified deniability by adding certified deniability to the Fiat-Shamir transformation.

A Composeable Definition. We also give a definition where the adversary only receives a single signature, but the definition has the ability to compose with itself. We give the definition in the QROM, but it can be straightforwardly moved to the plain model.

Define the following security game $\text{Sig-CDen-1}_{\text{Adv}(\text{R}_{\text{Adv}})}^H(\text{sk}, \text{vk}, m)$:

1. Sample $(|\sigma\rangle, \text{dk}) \leftarrow \text{Sign}(\text{sk}, m)$ and send $|\sigma\rangle$ to the adversary.
2. The adversary outputs a certificate register R_{cert} and an output register R_{out} .
3. If $\text{DelVer}(\text{dk}, \text{R}_{\text{cert}}) = \text{Accept}$, output R_{out} . Otherwise output $\perp$.

Definition 9 (Composeable Certified Deniability for Signatures). *A revocable signature scheme $(\text{Gen}, \text{Sign}, \text{Verify}, \text{Del}, \text{DelVer})$ has composeable certified deniability in the quantum random oracle model if there exists a QPT simulator $\mathcal{S}$ such that for every QPT adversary Adv with poly-size auxiliary input register R_{Adv} , every message m , and every (sk, vk) in the support of $\text{Gen}(1^\lambda)$,*

$$\left\{ (O_H, \text{vk}, \text{Sig-CDen-1}_{\text{Adv}(\text{R}_{\text{Adv}})}^H(\text{sk}, \text{vk}, m)) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \\ \approx_c \\ \left\{ (O_H, \text{vk}, S^H(\text{Adv}, \text{R}_{\text{Adv}}, \text{vk}, m)) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

where O_H denotes oracle access to H and M_S is the set of messages on which $\mathcal{S}$ queries $\text{Sign}^H(\text{sk}, \cdot)$.

There are a few main differences from [Definition 8](#). First, the adversary only ever gets one signature, which they have promised to delete. Accordingly, the simulator gets *no* signatures/does not get access to a signing oracle. Second, the difference that makes this definition composeable is the quantification over all vk , rather than requiring that security holds over a randomly generated vk . As such, we can rely on deletion even when the adversary already holds information that depends on vk — such as another signature. This quantification over all vk is possible because unforgeability is defined *separately* from certified deniability. More formally, we show that the composeable definition implies the selective definition which explicitly gives multiple signatures to the adversary.

Property 1 (Composeability). *Any revocable signature scheme with composeable certified deniability ([Definition 9](#)) also has selective certified deniability ([Definition 8](#)). This holds both in the QROM and in the plain model.*

Proof. Let $\mathcal{S}_+$ be the simulator for the composeable definition. We show how to build a simulator $\mathcal{S}_s$ for the selective definition.

Any adversary Adv for the selective definition makes at most $q = \text{poly}[\lambda]$ queries to the signing oracle. We define a series of q hybrid adversaries Adv_i which do not perform queries $\leq i$ which it intends to delete.

- Hyb_0 is the selective experiment run on the following adversary Adv_0 , except omitting the final check that D matches the pre-declared multi-set of messages. It still includes the check that Adv presents valid certificates for queries it declared that it would delete.

Adv_0 has sk hard-coded. It runs $\text{Adv}(\text{R}_{\text{Adv}}, \text{vk})$. During this, it answers all of Adv 's queries using sk . Then, it checks the deletion certificates for the declared queries and outputs $\perp$ if any are rejected.

- Hyb_i is the selective experiment run on the following adversary Adv_i , except omitting the final check that D matches the pre-declared multi-set of messages. It still includes the check that Adv presents valid certificates for queries it declared that it would delete.

$\text{Adv}_i^{\text{Sign}(\text{sk}, \cdot)}$ has sk hard-coded and also has query access to $\text{Sign}(\text{sk}, \cdot)$. It runs $\text{Adv}_{i-1}^{\text{Sign}(\text{sk}, \cdot)}$ until Adv_{i-1} submits query i for m_i . Let $\text{Adv}_{i-1}[i :]$ be the rest of its code and R_{i-1} contain its current state. If Adv_{i-1} declares that it will delete the result of query i , run $\mathcal{S}_+^H(\text{Adv}_{i-1}[i], \text{R}_{i-1}, m_i)$ by internally answering all of its queries to the signing oracle using sk . Otherwise, forward the i 'th query to $\text{Sign}(\text{sk}, \cdot)$ and continue running Adv_{i-1} while answering all further queries using sk instead of directly querying $\text{Sign}(\text{sk}, \cdot)$.

Note that sk is *only* used for queries $\geq i + 1$.

- $\mathcal{S}_s^{\text{Sign}(\text{sk}, \cdot)}(\text{Adv})$ is the same as Adv_q , except for the following changes. First, it does not use a hardcoded sk , since Adv_q does not require it either. Second, at the end of the game it checks that the pre-declared multi-set of messages matches the signatures for which it received a valid certificate; if not, it outputs $\perp$ instead of its view.

Adv_0 's output is clearly identical to running the selective experiment with $\text{Adv}^{\text{Sign}(\text{sk}, \cdot)}(\text{R}_{\text{Adv}}, \text{vk})$ and omitting the final check that D matches the pre-declared multi-set of messages. Observe that if the composeable experiment on query i would abort for Adv_i , then the selective experiment would also abort for Adv_i . Therefore Hyb_i is indistinguishable from Hyb_{i-1} by the composeable security of the revocable signature scheme. Finally, $\mathcal{S}_s^{\text{Sign}(\text{sk}, \cdot)}(\text{Adv})$ matches the full selective experiment by adding the check that D matches the pre-declared multi-set of messages.

The proof is essentially identical for the plain model. $\square$

4.2 NIZKs

Previously, we saw a composeable definition for signatures which implies selective certified deletion when receiving multiple signatures. Since NIZKs are similar to signatures, we directly define certified deniability in the composeable style. If one were to define a similar multi-proof definition for NIZKs, then the following definition would imply the selective version of it in the same manner as for signatures.

Certified Deniability: Real Experiment. Certified deniability follows the standard real/ideal paradigm of simulator-based definitions. The real experiment $\text{NIZK-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}(s, w)$ is parameterized by an adversarial QPT algorithm with auxiliary input R_{Adv} and some NP statement and witness pair (s, w) . $\text{NIZK-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}(s, w)$ consists of the following distribution:

1. Sample $(|\pi\rangle, \text{dk}) \leftarrow \text{Prove}(s, w)$ and send $|\pi\rangle$ to the adversary.
2. The adversary outputs two registers $(\text{R}_{\text{cert}}, \text{R}_{\text{Adv}}) \leftarrow \text{Adv}(\text{R}_{\text{Adv}}, |\pi\rangle)$.

3. If $\text{DelVer}(\text{dk}, \text{R}_{\text{cert}})$ outputs accept, then the experiment outputs the adversary's residual state register R_{Adv} . Otherwise, it outputs $\perp$.

If working in an oracle model where the parties have access to an oracle H , then we denote the experiment as $\text{NIZK-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}^H(s, w)$.

Definition 10 (Certified Deniability for NIZKs: Plain Model). *A revocable non-interactive argument system (see Section 3.3) $(\text{Prove}, \text{Verify}, \text{DelVer})$ is certifiably deniable if there exists a QPT simulator $\mathcal{S}$ such that for every QPT adversary Adv with poly-size auxiliary input register R_{Adv} and every NP statement/witness pair (s, w) ,*

$$\{\text{NIZK-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}(s, w)\} \approx_c \{\mathcal{S}(s, \text{Adv}, \text{R}_{\text{Adv}})\}$$

We may also define certified deniability in the CRS model or the QROM model. Following [Pas03]'s definition from the classical setting, a deniable simulator does *not* have the ability to program the global random oracle; this is enforced by sampling a fresh random oracle before the experiment and including its description in the output of the experiment. Thus, any simulator which attempts to pretend that the oracle had different behavior is likely to be caught.

Definition 11 (Certified Deniability for NIZKs: QROM). *A revocable non-interactive argument system $(\text{Prove}, \text{Verify}, \text{DelVer})$ is certifiably deniable in the quantum random oracle model if there exists a QPT simulator $\mathcal{S}$ such that for every QPT adversary Adv with poly-size auxiliary input register R_{Adv} and every NP statement/witness pair (s, w) ,*

$$\begin{aligned} & \left\{ \left(\text{NIZK-CDen}_{\text{Adv}(\text{R}_{\text{Adv}})}^H(s, w), O_H \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \\ & \approx_c \\ & \left\{ \left(\mathcal{S}^H(s, \text{Adv}, \text{R}_{\text{Adv}}), O_H \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \end{aligned}$$

where O_H denotes oracle access to H .

We show in Section 5 how to obtain NIZKs with certified deniability by adding certified deniability to the Fiat-Shamir transformation.

5 Fiat-Shamir with Certified Deniability

In this section, we show how to modify the Fiat-Shamir transform to add certified deniability. As a result of this general paradigm, we obtain both signatures with certified deniability (see Section 4.1) and non-interactive zero knowledge with certified deniability (see Section 4.2).

Let Σ be a Sigma protocol, and denote FS_{Σ}^H as its Fiat-Shamir transform with oracle access to H . We denote an oracle with the first part of the input fixed to v as $H(v\|\cdot)$. On query w , it returns $H(v\|w)$.

Construction 1 (NIZK with Certified Deniability). *The construction, which we call FS-CDen_{Σ} for easy reference later, is as follows.*

- Prove (x, w) :
 1. Sample a subspace $A \subset \{0, 1\}^{\lambda}$ of dimension $\lambda/2$ and an offset $s \leftarrow \{0, 1\}^{\lambda}$, then prepare the coset state $|A_{0,s}\rangle \propto \sum_{a \in A} (-1)^{a \cdot s} |a\rangle$ in register R_A . Initialize register $\text{R}_{\Sigma} = (\text{R}_{\Sigma,1}, \text{R}_{\Sigma,2}, \text{R}_{\Sigma,3})$ to $|0\rangle$.

2. Sample a key $k \leftarrow \{0, 1\}^\lambda$ and apply the isometry

$$|a\rangle_{R_A} \otimes |\vec{0}\rangle_{R_\Sigma} \mapsto |a\rangle_{R_A} \otimes \left| \text{FS}_\Sigma^{H(a|\cdot)}(x, w; H(k||a)) \right\rangle_{R_\Sigma}$$

to registers R_A and R_Σ . This results in a state

$$|\pi\rangle : \propto \sum_{a \in A} (-1)^{s \cdot a} |a\rangle_{R_A} \otimes |s_1^a, s_2^a, s_3^a\rangle_{R_\Sigma}$$

where $s_2^a = H(a||s_1^a)$ and (s_1^a, s_3^a) are the prover's messages from Σ using randomness $H(k||a)$.

3. Output $|\pi\rangle$ as the argument and $\text{dk} := (A, k, w, s)$ as the deletion key.

• Verify(x, R_{arg}):

1. Parse $R_{\text{arg}} = (R_A, R_\Sigma)$.

2. Coherently evaluate $\text{FS}_\Sigma.\text{Verify}$ on register R_Σ , then measure and output the result.

• Del(R_{arg}): Output R_{arg} .

• DelVer($\text{dk}, R_{\text{cert}}$):

1. Parse $\text{dk} = (A, k, w)$. Parse $R_{\text{arg}} = (R_A, R_\Sigma)$.

2. Apply the isometry

$$|a\rangle_{R_A} \otimes |y\rangle_{R_\Sigma} \mapsto |a\rangle_{R_A} \otimes \left| y \oplus \text{FS}_\Sigma^{H(a|\cdot)}(x, w; H(k||a)) \right\rangle_{R_\Sigma}$$

3. Measure register R_A with respect to the PVM $\{|A_{0,s}\rangle\langle A_{0,s}|, I - |A_{0,s}\rangle\langle A_{0,s}|\}$. Output accept if the result is the former, and reject if it is the latter.

Theorem 5. If Σ is a $2^{-\lambda}$ -secure sigma protocol for a language $\mathcal{L}$ with (1) perfect completeness and (2) quantum proof of knowledge, then FS-CDen_Σ ([Construction 1](#)) is a NIZKPoK for $\mathcal{L}$ with composable certified deniability in the QROM.

Remark: As mentioned in [Section 3.2](#), such sigma protocols exist unconditionally in the QROM.

Proof. We prove certified deniability in [Claim 1](#). Correctness of deletion follows from inspection and the fact that $|A_{0,s}\rangle\langle\{0\}|$ is negligibly close in trace distance to $|A_{0,s}\rangle$.

By [Lemma 5](#), any NIZK $|\pi\rangle$ is perfectly identical, from the verifier's point of view, from whether the first register R_A has been measured in the computational basis. Thus, [Construction 1](#) is sound and zero-knowledge if sampling a random a and outputting $a||\text{FS}_\Sigma^{H(a|\cdot)}(x, w)$ is sound and zero-knowledge. Observe that a can be considered to be part of the first message of underlying sigma protocol, which makes the sigma protocol have unpredictable first messages. Then [Theorem 3](#) implies that $a||\text{FS}_\Sigma^{H(a|\cdot)}(x, w)$ is a NIZKPoK, which implies it is sound and zero-knowledge. $\square$

[Construction 1](#) is already a NIZK, if instantiated with an appropriate sigma protocol. To obtain a signature scheme, a few more details are needed.

Construction 2 (Signature with Certified Deniability). Let $f : \{0, 1\}^\lambda \mapsto \{0, 1\}^{\text{poly}(\lambda)}$ be a one-way function and let Σ be a sigma protocol for the language

$$\mathcal{L}_f = \left\{ x \in \{0, 1\}^{\text{poly}(\lambda)} : \exists w \in \{0, 1\}^\lambda \text{ s.t. } f(w) = x \right\}$$

- Gen(1^λ) : Sample $w \leftarrow \{0, 1\}^\lambda$. Output $\text{vk} = f(w)$ and $\text{sk} = w$.
- Sign(sk, m) : Evaluate $(|\pi\rangle, \text{dk}) \leftarrow \text{FS-CDen}_\Sigma.\text{Prove}^{H(m||\cdot)}(\text{vk}, \text{sk})$ and output the result.
- Verify($\text{vk}, m, \text{R}_{\text{arg}}$) : Evaluate $\text{FS-CDen}_\Sigma.\text{Verify}^{H(m||\cdot)}(\text{vk}, \text{R}_{\text{arg}})$.
- Del(R_{arg}) : Output $\text{FS-CDen}_\Sigma.\text{Del}^{H(m||\cdot)}(\text{R}_{\text{arg}})$.
- DelVer($\text{dk}, \text{R}_{\text{cert}}$) : Output $\text{FS-CDen}_\Sigma.\text{DelVer}^{H(m||\cdot)}(\text{dk}, \text{R}_{\text{cert}})$.

Theorem 6. If f is a one-way function and Σ is a collapsing, $2^{-\lambda}$ -secure sigma protocol for $\mathcal{L}_f$, then **Construction 2** is a signature scheme with certified deniability in the QROM.

Proof. We prove certified deniability in **Claim 1**. Correctness of deletion follows from inspection and the fact that $|A_{0,s}\setminus\{0\}\rangle$ is negligibly close in trace distance to $|A_{0,s}\rangle$.

By **Lemma 5**, any NIZK $|\pi\rangle$ is perfectly identical, from the verifier's point of view, from whether the first register R_A has been measured in the computational basis. Thus, **Construction 1** is existentially unforgeable if sampling a random a and outputting $a||\text{FS}_\Sigma^{H(m||a||\cdot)}(x, w)$ is existentially unforgeable. Observe that a can be considered to be part of the first message of the underlying sigma protocol without affecting its properties. Then **Theorem 4** implies that $a||\text{FS}_\Sigma^{H(m||a||\cdot)}(x, w)$ is existentially unforgeable. $\square$

5.1 Proof of Certified Deniability

Claim 1. If Sigma is a $2^{-\lambda}$ -secure sigma-protocol for a language $\mathcal{L}$, then **Construction 1** satisfies certified deniability for NIZKs in the QROM (**Definition 11**). Furthermore, assuming the same, **Construction 2** satisfies composable certified deletion for signatures in the QROM (**Definition 8**).

Proof. The proofs of the two sub-claims are almost identical, except for a slight difference in the simulators. The NIZK simulator $\mathcal{S}$ uses statement x and is given below. The signature simulator uses the verification key vk instead of x , and additionally forwards any adversarial queries for signatures to the signing oracle, except if the adversary would query for m^* . Furthermore, in the case of signatures, we regard $H(m||\cdot)$ to be the random oracle, so we omit explicitly prepending m to the random oracle queries below. In both, we consider the verification key vk (respectively, the statement x) to be fixed, as required for the composable definition. For the rest of this proof, we focus on the signature case.

On input a statement x , the adversary's code Adv and auxiliary input register R_{Adv} , the simulator $\mathcal{S}$ works as follows:

1. Sample a subspace $A \subset \{0, 1\}^\lambda$ of dimension $\lambda/2$ and sample an offset $s \leftarrow \{0, 1\}^\lambda$, then prepare the coset state $|A_{0,s}\setminus\{0\}\rangle \propto \sum_{a \in A \setminus \{0\}} (-1)^{a \cdot s} |a\rangle$ in register R_A . Initialize register $\text{R}_\Sigma = (\text{R}_{\Sigma,1}, \text{R}_{\Sigma,2}, \text{R}_{\Sigma,3})$ to $|0\rangle$.
2. Sample keys $k_{\text{ch}}, k \leftarrow \{0, 1\}^\lambda$. Apply the isometry

$$|a\rangle_{\text{R}_A} \otimes |0\rangle_{\text{R}_\Sigma} \mapsto |a\rangle_{\text{R}_A} \otimes |\mathcal{S}_\Sigma(x, H(k_{\text{ch}}||a); H(k||a))\rangle_{\text{R}_\Sigma}$$

to registers R_A and R_Σ .

$$|\tilde{\pi}\rangle : \propto \sum_{a \in A \setminus \{0\}} (-1)^{a \cdot s} |a\rangle_{\text{R}_A} \otimes \left| \tilde{s}_1^a, \tilde{s}_2^a, \tilde{s}_3^a \right\rangle_{\text{R}_\Sigma}$$

where $\tilde{s}_2^a = H(k_{\text{ch}}\|a)$ and $(\tilde{s}_1^a, \tilde{s}_3^a)$ are obtained by running the honest verifier zero knowledge simulator $\mathcal{S}_\Sigma$ for Σ .

3. Define the random oracle H' as follows:

$$H'(q_1\|q_2\|q_3) := \begin{cases} H(k_{\text{ch}}\|q_1) & \text{if } q_1 \in A \setminus \{0\} \text{ and } q_2\|q_3 = x\|\tilde{s}_1^{q_1} \\ H(q_1\|q_2\|q_3) & \text{else.} \end{cases} \quad (1)$$

where $(\tilde{s}_1^a, \tilde{s}_2^a, \tilde{s}_3^a) = \mathcal{S}_\Sigma(x, H(k_{\text{ch}}\|a); H(k\|a))$ for all $a \in A \setminus \{0\}$. Note that this may be efficiently evaluated on any computational basis queries using the description of A , and thus on any quantum queries in general.

4. Compute the adversary's output $(R_{\text{cert}}, R_{\text{Adv}}) \leftarrow \text{Adv}^{H'}(R_{\text{Adv}}, |\tilde{\pi}\rangle)$.
5. Parse $R_{\text{cert}} = (R_A, R_\Sigma)$ and compute the isometry

$$|a\rangle_{R_A} \otimes |y\rangle_{R_\Sigma} \mapsto |a\rangle_{R_A} \otimes |y \oplus \mathcal{S}_\Sigma(x, H(k_{\text{ch}}\|q_2); H(k\|a))\rangle_{R_\Sigma}$$

on registers R_A and R_Σ .

6. Measure register R_A with respect to the PVM $\{|A_{0,s}\rangle\langle A_{0,s}|, I - |A_{0,s}\rangle\langle A_{0,s}|\}$. If the result is the former, output R_{Adv} . Otherwise output $\perp$.

We now show that this simulator satisfies [Definition 11](#), i.e. the joint distribution over the output of the simulator and the description of H is indistinguishable from the real certified deniability experiment. Consider the following hybrid experiments:

- Hyb₀(w): The real certified deniability experiment $\text{NIZK-CDen}_{\text{Adv}(R_{\text{Adv}})}^H(x, w)$. Note that this does not encompass the whole output distribution from the certified deniability definition in the QROM ([Definition 8](#)); it is missing the O_H output.
- Hyb₁(w): The only difference from Hyb₀ is that we replace the oracle H with a reprogrammed oracle H'_1 , defined as⁹

$$H'_1(q_1\|q_2\|q_3) := \begin{cases} H(k_{\text{ch}}\|q_1) & \text{if } q_1 \in A \setminus \{0\} \text{ and } q_2\|q_3 = x\|\tilde{s}_1^{q_1} \\ H(q_1\|q_2\|q_3) & \text{else.} \end{cases} \quad (2)$$

where $s_1^a = P_{\Sigma,1}((x, w); H(k\|a))$.¹⁰ In the security game $\text{Hyb}_1(w) = \text{NIZK-CDen}_{\text{Adv}(R_{\text{Adv}})}^{H'_1}(x, w)$, this results in a NIZK

$$|\pi\rangle \propto \sum_{a \in A \setminus \{0\}} (-1)^{a \cdot s} |a\rangle \otimes |s_1^a, \tilde{s}_2^a, s_3^a\rangle$$

where $\tilde{s}_2^a = H(k_{\text{ch}}\|a)$ and (s_1^a, s_3^a) are computed honestly using the witness w .

We briefly comment on the relation between H' , which is used by the simulator, and H'_1 . Both oracles can be viewed as the same parameterized oracle, where the parameters are every s_1^a . In H'_1 , the s_1^a are honestly generated, whereas in H' they are generated by the HVZK simulator for Σ .

⁹Note that queries to H'_1 can be answered lazily, preventing an exponential blow-up from reprogramming an exponential number of positions. Since H'_1 is implicitly pre-defined in terms of H , which is fixed, this does *not* require maintaining state, e.g. through the compressed oracle technique.

¹⁰We abstract the expansion of s_1^a out of the definition of H'_1 to emphasize that s_1^a is treated as an implicitly defined parameter to the reprogrammed oracle.

- Hyb₂ = S: The only differences from Hyb₁ are for each $a \in A \setminus \{0\}$, we replace (s_1^a, s_3^a) with an honest-verifier simulated transcript $(\tilde{s}_1^a, \tilde{s}_3^a)$ from

$$(\tilde{s}_1^a, \tilde{s}_2^a, \tilde{s}_3^a) = \mathcal{S}_\Sigma(x, H(k_{\text{ch}} \| a); H(k \| a))$$

and update the random oracle accordingly, as defined in Equation (2) with $s_1^a = \tilde{s}_1^a$. This results in the oracle H' .

In the security game, this modification results in a NIZK

$$|\tilde{\pi}\rangle \propto \sum_{a \in A \setminus \{0\}} (-1)^{a \cdot s} |a\rangle \otimes |\tilde{s}_1^a, \tilde{s}_2^a, \tilde{s}_3^a\rangle$$

Note that the definition of $(\tilde{s}_1^a, \tilde{s}_3^a)$ is implicit, and they are only computed coherently as necessary: (1) in answering queries to H'_2 , (2) in computing $|\tilde{\pi}\rangle$, and (3) in applying the isometry $|a\rangle \otimes |y\rangle \mapsto |a\rangle \otimes |y \oplus (\tilde{s}_1^a, \tilde{s}_2^a, \tilde{s}_3^a)\rangle$ to check the deletion certificate.

We now show in Claims 2 to 4 that

$$\begin{aligned} (\text{Hyb}_0, O_H) &\approx_c (\text{Hyb}_1, O_{H'_1}) \\ &\approx_c (\text{Hyb}_2, O_{H'}) \\ &\approx_c (\text{Hyb}_2, O_H) \end{aligned}$$

over the choice of the random oracle H .

Claim 2.

$$\{(\text{Hyb}_0, O_H) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\} \approx \{(\text{Hyb}_1, O_{H'_1}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\}$$

Proof. Consider the sub-hybrid experiment Hyb'_0 :

- Hyb'₀: This hybrid is identical to Hyb₀, except that H is replaced with a reprogrammed oracle H'_0 , defined as

$$H'_0(v) := \begin{cases} G(v') & \text{if } v = k_{\text{ch}} \| v' \text{ for some } v' \\ H(v) & \text{else.} \end{cases}$$

where $\mathcal{X} = \{0, 1\}^\lambda \times \mathcal{X}_2$ and $G \leftarrow \text{Func}(\mathcal{X}_2, \mathcal{Y})$ is a fresh random oracle.

- Hyb₁: The only difference between Hyb'₀ and Hyb₁ is the challenger samples a key $k_{\text{ch}} \leftarrow \{0, 1\}^\lambda$ and G is defined by $G(v) = H(k_{\text{ch}} \| v)$.

Since G is a truly random function and is only used in the definition of H'_0 ,

$$\{(\text{Hyb}_0, O_H) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\} = \{(\text{Hyb}'_0, O_{H'_0}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\}$$

Furthermore, Lemma 4 implies¹¹

$$\{(\text{Hyb}'_0, O_{H'_0}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\} \approx_c \{(\text{Hyb}_1, O_{H'_1}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})\}$$

□

¹¹Since G is used to define H'_0 , changing G to match $H(k_{\text{ch}} \| \cdot)$ causes H'_0 to become H'_1 .

Claim 3. If Σ is a sigma-protocol for a language $\mathcal{L}$ with $2^{-\lambda}$ -security, where $\nu(\lambda)$ is a negligible function, then

$$\left\{ \left(\text{Hyb}_1, O_{H'_1} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \approx \left\{ \left(\text{Hyb}_2, O_{H'} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

Proof. Let a_j be the j 'th lexicographically least element of A . We define a series of $2 \cdot |A| = 2 \cdot 2^{\lambda/2}$ intermediate hybrid experiments $\text{Hyb}_{1,i,1}$ and $\text{Hyb}_{1,i,2}$ for $i \in [|A|]$, as well as an additional hybrid experiment Hyb'_1 .

- Hyb'_1 : This hybrid is identical to Hyb_1 , except that the challenger uses an additional random oracle $G \leftarrow \text{Func}(\{0,1\}^\lambda, \mathcal{Y})$. Whenever it would evaluate $H(k||v)$ for some v , it instead (coherently) evaluates $G(v)$.
- $\text{Hyb}_{1,i,1}$: This is identical to Hyb'_1 , except for the following changes. First, for every $j < i$, the honestly computed $(s_1^{a_j}, s_3^{a_j})$ are replaced by $(\widetilde{s}_1^{a_j}, \widetilde{s}_3^{a_j})$, which are generated by $\mathcal{S}_\Sigma$ using randomness $G(a_j)$. Since $s_1^{a_j}$ is modified, the random oracle is also updated accordingly to become $H'_{1,i}$, as defined by Equation (2) with $s_1^{a_j} := \widetilde{s}_1^{a_j}$ for $j < i$.

The second difference is purely syntactic. Instead of getting access to G , the challenger gets access to G_{a_i} , which is identical to G except that $G_{a_i}(a_i) = \perp$. Additionally, it receives a classical copy of $(s_1^{a_i}, s_3^{a_i})$, which are computed by the honest Σ prover using (x, w) and randomness $G(a_i)$.

- $\text{Hyb}_{1,i,2}$: This is identical to $\text{Hyb}_{1,i,1}$, except that the honestly computed $(s_1^{a_i}, s_3^{a_i})$ are replaced by $(\widetilde{s}_1^{a_i}, \widetilde{s}_3^{a_i})$, where

$$(\widetilde{s}_1^{a_i}, \widetilde{s}_2^{a_i}, \widetilde{s}_3^{a_i}) = \mathcal{S}_\Sigma(x, H(k_{\text{ch}}||a_i); G(a_i))$$

- Hyb_2 : The only difference between Hyb_2 and $\text{Hyb}_{1,i_{\max},2}$, where $i_{\max} = 2^{\lambda/2}$, is the challenger samples a key $k \leftarrow \{0,1\}^\lambda$ and G is defined by $G(v) = H(k||v)$.

Lemma 4 implies

$$\left\{ \left(\text{Hyb}_1, O_{H'_1} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \approx_c \left\{ \left(\text{Hyb}'_1, O_{H'_1} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

Observe that $\text{Hyb}_{1,0,1}$ makes only syntactic changes from Hyb'_1 , so for all H ,

$$\text{Hyb}'_1 = \text{Hyb}_{1,0,1}$$

The same observation holds for every $\text{Hyb}_{1,i,2}$ and $\text{Hyb}_{1,i+1,1}$, so for all H ,

$$\text{Hyb}_{1,i,2} = \text{Hyb}_{1,i+1,1}$$

We now show that¹²

$$\left\{ \left(\text{Hyb}_{1,i,1}, O_{H'_{1,i}} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \approx_c^{2^{-\lambda}} \left\{ \left(\text{Hyb}_{1,i,2}, O_{H'_{1,i+1}} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

by reducing to the $2^{-\lambda}$ -security of Σ . The reduction simulates the random oracles H and G_{a_i} using the compressed oracle technique [Zha19],¹³ then declares $H(k_{\text{ch}}||a_i)$ as its challenge $s_2^{a_i}$ in an

¹²As before, $H_{1,i}$ is defined using $s_1^{a_i}$, so modifying it to become simulated changes $H'_{1,i}$ to $H'_{1,i+1}$.

¹³If Σ is HVZK in the QROM, e.g. because it is designed for the QROM, then this step is unnecessary.

honest-verifier execution of Σ . It receives either $(s_1^{a_i}, s_3^{a_i})$ generated by an honest prover, or $(\widetilde{s}_1^{a_i}, \widetilde{s}_3^{a_i})$ generated by the HVZK simulator $\mathcal{S}_\Sigma$. Using these, it can compute the rest of the experiment according to the description of $\text{Hyb}_{1,i,1}$. In the former case, the resulting distribution is $\text{Hyb}_{1,i,1}$; in the latter, it is $\text{Hyb}_{1,i,2}$. Therefore, if the two distributions above were distinguishable with advantage ϵ , then applying that same distinguisher would distinguish an honest Σ prover's messages from the output of the honest-verifier simulator $\mathcal{S}_\Sigma$ also with advantage ϵ . The $2^{-\lambda}$ -security of Σ implies that $\epsilon \leq 2^{-\lambda}$ for all QPT distinguishers.

Lemma 4 implies

$$\left\{ \left(\text{Hyb}_{1,i_{\max},2}, O_{H'_{1,i+1}} \right) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \approx_c \left\{ (\text{Hyb}_2, O_{H'}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

By the triangle inequality, the distinguishing advantage between $\left\{ (\text{Hyb}_1, O_{H'_1}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$ and $\left\{ (\text{Hyb}_2, O_{H'}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$ for any QPT distinguisher is bounded by $\text{negl}(\lambda) + 2^{\lambda/2} \cdot 2^{-\lambda} + \text{negl}(\lambda) = \text{negl}(\lambda)$. $\square$

Claim 4.

$$\left\{ (\text{Hyb}_2, O_{H'}) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\} \approx \left\{ (\text{Hyb}_2, O_H) : H \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y}) \right\}$$

Proof. We reduce to the direct product hardness of subspace states (**Lemma 2**). Assume, for the sake of contradiction, that some QPT distinguisher D with auxiliary input register R_D were able to distinguish between these two distributions.

Before describing the reduction, we claim that D has noticeable advantage conditioned on Hyb_2 not outputting $\perp$, i.e. conditioned on the simulator's certificate check passing. We also claim that Hyb_2 outputs $\perp$ with at most $1 - 1/p$ probability for some polynomial p . We reduce these claims to direct product hardness (**Lemma 2**). The reduction takes in a subspace state $|A\rangle$ and membership oracles $O_A, O_{A^\perp}$. If either of the claimed conditions do not hold, then D must have noticeable advantage conditioned on Hyb_2 outputting $\perp$. Consider running $D(\perp, O_{H'})$, which can be done without using $|A\rangle$. By **Lemma 3**, measuring one of D 's queries at random produces a point x^* where $H(x^*) \neq H'(x^*)$ with noticeable probability. These are exactly values of the form $a\|b$ for $a \in A \setminus \{0\}$. Since H' can be implemented using O_A , this procedure finds an element of A with noticeable probability using only a polynomial number of queries to O_A , without using $|A\rangle$. Combining this with a measurement of $|A\rangle$ in the Hadamard basis, the reduction obtains a pair of vectors in $A \setminus \{0\} \times A^\perp \setminus \{0\}$ with noticeable probability, breaking direct product hardness.

Next, we show that D cannot have noticeable distinguishing advantage when conditioned on Hyb_2 not outputting $\perp$. We again reduce to the direct product hardness of coset states. The reduction takes as input a random subspace state $|A\rangle$ and oracle access to membership oracles $O_A, O_{A^\perp}$. It runs the simulator (i.e. Hyb_2) using $|A\rangle$. Whenever it would check that a vector v is in A (respectively, $A^\perp$), it queries v to O_A (respectively, $O_{A^\perp}$). To implement step 6 (the certificate check), it first applies the following operations to register R_A : (1) a Hadamard operation, (2) the isometry $|y\rangle \mapsto |y - s\rangle$, and (3) a Hadamard operation. Then, it uses O_A and $O_{A^\perp}$ to implement the PVM $\{|A\rangle\langle A|, I - |A\rangle\langle A|\}$ on register R_A , as described by **Lemma 1**. If the result of the simulator is $R_{\text{Adv}} \neq \perp$, then it runs the distinguisher $D^{H'}(R_D, R_{\text{Adv}})$ and measures a random query that D makes to H' . Let $q = q_1\|q_2$ be the query measurement result. Finally, the reduction measures R_A in the Hadamard basis to obtain a value v and outputs (q_1, v) .

Recall that Hyb_2 outputs $R_{\text{Adv}} \neq \perp$ with noticeable probability. Condition on this case occurring. This happens exactly when the measurement on R_A returns result $|A\rangle\langle A|$, in which case

R_A collapses to $|A\rangle$. Therefore the measurement result $v \in A^\perp$ and is not 0 with overwhelming probability. Furthermore, we previously established that D has noticeable advantage in this case; therefore [Lemma 3](#) implies that $H(q_1\|q_2) \neq H'(q_1\|q_2)$ with noticeable probability. Whenever this occurs, $q_1 \in A$. Therefore $(q_1, v) \in A \setminus \{0\} \times A^\perp \setminus \{0\}$ with noticeable probability, violating [Lemma 2](#). $\square$

$\square$

6 Negative Results

In this section, we give evidence that certifiable deniability is in fact *impossible* in the plain model. Specifically, we show a black-box barrier against obtaining signatures with certified deniability in the plain model; any work achieving this must use non-black-box techniques in their security proof.

6.1 Supporting Lemma.

Before proving the main result of this section, we introduce a supporting lemma that generalizes a result from [\[BBBV97\]](#) about reprogramming quantum-accessible oracles. This lemma may be of independent interest.

In their original version, they consider a quantum adversary who has quantum query access to one of two classical oracles H and H' . They bound the ability of the adversary to distinguish between the two oracles in terms of the amplitude with which it queries on (classical) inputs x where $H(x) \neq H'(x)$. As a simple corollary, if the adversary is able to distinguish the two oracles in a polynomial number of queries, then measuring one of its queries at random produces an x such that $H_0(x) \neq H_1(x)$ with noticeable probability.

We observe that a similar statement holds for quantum oracles, i.e. oracles which take in a quantum input, perform quantum computation, and produce quantum output. If the adversary is able to distinguish the two oracles in a polynomial number of queries, then outputting the query register at a random query produces a mixed state with noticeable probability mass on states $|\psi\rangle$ where $H(|\psi\rangle)$ is far from $H'(|\psi\rangle)$.

Lemma 6. *Let H and H' be oracle-accessible unitaries and let $A^{(\cdot)}$ be a quantum oracle algorithm with auxiliary input $|\psi_0\rangle$. Let*

$$|\psi_t\rangle = \sum_i \alpha_{t,i} |\phi_{t,i}\rangle_{R_A} \otimes |q_{t,i}\rangle_{R_Q}$$

be a Schmidt decomposition of the state of A^H on submitting query t , where R_A is the internal register of A and R_Q is the query register. Let ψ'_t similarly be the state of $A^{H'}$ on submitting query t . Then for all $T \in \mathbb{N}$,

$$\text{TD}[|\psi_{T+1}\rangle\langle\psi_{T+1}|, |\psi'_{T+1}\rangle\langle\psi'_{T+1}|] \leq \sqrt{4T \sum_{t=1}^T \sum_i |\alpha_{t,i}|^2 \text{TD}[H(|q_{t,i}\rangle), H'(|q_{t,i}\rangle)]^2}$$

Proof. The oracle algorithm A^H can be expressed as sequence of unitaries A_t interleaved with unitary oracle operations H , i.e.

$$|\psi_{T+1}\rangle = A_{T+1} \left(\prod_{t=1}^T H A_t \right) |\psi_0\rangle$$

Note that $|\psi_{t+1}\rangle$ is prepared by applying A_{t+1} to the result $H|\psi_t\rangle$ of the prior query. Define $|E_t\rangle$ to be the error term resulting from answering the t 'th query of A^H using H' instead of H , i.e.

$$|E_t\rangle = H'|\psi_t\rangle - H|\psi_t\rangle$$

Then we can express the state of $A^H(|\psi_0\rangle)$ on submitting the $(T+1)$ 'th query (before it is answered) as

$$|\psi_{T+1}\rangle = A_T H |\psi_T\rangle \quad (3)$$

$$= A_T (H' |\psi_T\rangle - |E_T\rangle) \quad (4)$$

Applying this decomposition recursively on $|\psi_T\rangle$ yields

$$|\psi_{T+1}\rangle = A_{T+1} \left(\prod_{t=1}^T H' A_t \right) |\psi_0\rangle - \sum_{t=1}^T \left(\prod_{i=t+1}^T H A_i \right) A_t |E_t\rangle \quad (5)$$

$$= |\psi'_{T+1}\rangle - \sum_{t=1}^T \left(\prod_{i=t+1}^T H A_i \right) |E_t\rangle \quad (6)$$

Thus we have

$$\begin{aligned} & \text{TD}[|\psi_{T+1}\rangle\langle\psi_{T+1}|, |\psi'_{T+1}\rangle\langle\psi'_{T+1}|] \\ &= \sqrt{1 - |\langle\psi_T|\psi'_T\rangle|^2} \end{aligned} \quad (7)$$

$$\leq \sqrt{1 - |\langle\psi_T|\psi_T\rangle + (\langle\psi'_T| - \langle\psi_T|)|\psi_T\rangle|^2} \quad (8)$$

$$\leq \sqrt{|\langle\psi'_T| - \langle\psi_T||\psi_T\rangle|^2} \quad (9)$$

$$\leq \sqrt{\|\psi'_T\rangle - |\psi_T\rangle\|_2^2} \quad (10)$$

$$= \sqrt{\left\| \sum_{t=1}^T \left(\prod_{i=t+1}^T H A_i \right) |E_t\rangle \right\|_2^2} \quad (11)$$

$$\leq \sqrt{T \sum_{t=1}^T \left\| \left(\prod_{i=t+1}^T H A_i \right) |E_t\rangle \right\|_2^2} \quad (12)$$

$$= \sqrt{T \sum_{t=1}^T \| |E_t\rangle \|_2^2} \quad (13)$$

$$= \sqrt{T \sum_{t=1}^T \| H' |\psi_t\rangle - H |\psi_t\rangle \|_2^2} \quad (14)$$

$$= \sqrt{T \sum_{t=1}^T \left| \sum_{i,j} \alpha_{t,i} \overline{\alpha_{t,j}} \langle \phi_{t,i} | \phi_{t,j} \rangle \langle q_{t,i} | (H - H')^\dagger (H - H') | q_{t,j} \rangle \right|^2} \quad (15)$$

$$\leq \sqrt{T \sum_{t=1}^T \sum_i |\alpha_{t,i}|^2 \| (H - H') | q_{t,i} \rangle \|_2^2} \quad (16)$$

$$\leq \sqrt{T \sum_{t=1}^T \sum_i |\alpha_{t,i}|^2 \|(H - H')|q_{t,i}\rangle\|_1^2} \quad (17)$$

$$= \sqrt{4T \sum_{t=1}^T \sum_i |\alpha_{t,i}|^2 \text{TD}[H(|q_{t,i}\rangle), H'|q_{t,i}\rangle]^2} \quad (18)$$

where Equation (10) follows from Cauchy-Schwarz; Equation (12) follows from Jensen's inequality; Equation (13) follows from the invariance of the ℓ_2 norm under unitaries; Equation (16) follows from Jensen's inequality. $\square$

Corollary 1 (One-Way to Hiding for Unitary Oracles). *Let H and H' be oracle-accessible unitaries and let $A^{(\cdot)}$ be a quantum oracle algorithm with auxiliary input $|\psi_0\rangle$ that uses at most T queries. Denote the mixed state resulting from measuring A^H 's query register at a random time t as*

$$\frac{1}{T} \sum_{t=1}^T |t\rangle\langle t| \otimes \sum_i |\alpha_{t,i}|^2 |q_{t,i}\rangle\langle q_{t,i}|$$

For $0 < \delta \leq 1$, denote the set S_δ as being the subset of (t, i) pairs such that $\text{TD}[H(|q_{t,i}\rangle), H'(|q_{t,i}\rangle)] \geq \delta$. If $A^{(\cdot)}$ distinguishes between H and H' with advantage ϵ in T queries, then

$$\frac{1}{T} \sum_{(t,i) \in S_\delta} |\alpha_{t,i}|^2 \geq \frac{\frac{\epsilon^2}{4T^2} - \delta^2}{1 - \delta^2}$$

If ϵ were noticeable, T were polynomial, and δ were set to $(1/2) \cdot \epsilon/(2T)$, then this immediately shows that the probability mass on pure states where H and H' differ by a noticeable trace distance is noticeable.

6.2 Plain Model Black-Box Barrier

For our black-box barrier, we model the simulator as having access to the adversary's unitary dilation and its inverse. However, we do not allow the simulator direct access to the adversary's internal registers, including its auxiliary input. As we discuss later, this is equivalent to a model where the simulator can directly access the internal registers, but they are ideally obfuscated.

Theorem 7. *There do not exist signatures with certified deniability in the plain model whose security proof makes only black-box use of the adversary's unitary dilation and its inverse.*

Remark 1. *We note that a similar statement holds for NIZKs with certified deniability as well. The proof is almost identical, so we omit it here.*

Proof. Let $\text{Sig-CDen} = (\text{Gen}, \text{Sign}, \text{Verify}, \text{Del}, \text{DelVer})$ be a candidate signature scheme with a black-box proof of certified deniability, i.e. which gives a simulator $\mathcal{S}$ that uses the adversary's unitary dilation as a black-box. We give a QPT adversary Adv together with a QPT distinguisher D that distinguishes $\mathcal{S}$ from the real experiment.

Adv receives as auxiliary input a post-quantum signing key $\tilde{k}$, which we denote as $\text{Adv}_{\tilde{k}}$. During the certified deniability game, $\text{Adv}_{\tilde{k}}$ receives a verification key vk , a state $|\sigma\rangle$, and a message m , then runs $\text{Verify}(\text{vk}, |\sigma\rangle, m)$, and if it accepts, outputs a post-quantum signature $\sigma_{m, \text{vk}}^{(\tilde{k})} \leftarrow \text{PQSig.Sign}(\tilde{k}, \text{vk} \| m)$ on the verification key concatenated with the message. It measures the result of this to get $\sigma_{m, \text{vk}}^{(\tilde{k})}$, which it writes to register R_{Adv} , then uncomputes the program.

By the correctness of **Sig-CDen**, this is gentle. Finally, it runs $\text{Del}(|\sigma_m\rangle)$ to obtain a valid deletion certificate in register R_{cert} and outputs R_{cert} along with R_{Adv} .

The distinguisher receives as auxiliary input the corresponding post-quantum verification key $\tilde{vk}$. On input R_{Adv} , it runs the post-quantum signature verification $\text{PQSig.Verify}(\tilde{vk}, R_{\text{Adv}})$ to check if Adv obtained a signature on $vk\|m$. If so, it outputs 1, and otherwise, outputs 0.

We now show that this adversary and distinguisher pair achieves noticeable advantage against any simulator $\mathcal{S}$. Observe that in the real certified deniability experiment, D outputs 1 with overwhelming probability, due to the correctness of the signature and obfuscation schemes. Conversely, we claim that against any QPT simulator, D outputs 0 with overwhelming probability.

Consider the following sequence of hybrid experiments:

- **Hyb₀**: The simulated certified deniability experiment. In more detail, sample $(sk, vk) \leftarrow \text{Gen}(1^\lambda)$. Run $\mathcal{S}^{\text{Adv}_{\tilde{k}}}$ to obtain an output register R_{Adv} .
- **Hyb₁**: This experiment is the same as **Hyb₀**, except that $\text{Adv}_{\tilde{k}}$ is replaced by $\text{Adv}_\perp$, which immediately honestly deletes the signature and always outputs $\perp$ in register R_{Adv} .

$\text{Hyb}_0 \approx \text{Hyb}_1$ from the unforgeability of the candidate signature scheme; otherwise, **Corollary 1** implies that outputting $\mathcal{S}^{\text{Adv}_{\tilde{k}}}$'s query register on a random query produces a mixed state with noticeable probability mass on an input $|\psi\rangle$ where $\text{Adv}_{\tilde{k}}(|\psi\rangle)$ and $\text{Adv}_\perp(|\psi\rangle)$ are noticeably far, which is exactly the set of points where $|\psi\rangle$ passes signature verification with noticeable probability.

We now claim that the original distinguisher $D(\tilde{vk})$ outputs 0 with overwhelming probability in **Hyb₁**. Otherwise, $\mathcal{S}^{\text{Adv}_\perp}$ must output a valid signature under $\tilde{vk}$, which it is independent of, which would violate the unforgeability of the post-quantum signature scheme. Since **Hyb₁** is indistinguishable from **Hyb₀** for all QPT distinguishers, $D(\tilde{vk})$ must also output 0 with overwhelming probability in **Hyb₀**, which is simply the output distribution of the certified deniability simulator. $\square$

Obfuscated Auxiliary Input. One could also consider implementing the adversary described above using an obfuscated program which Adv receives as auxiliary input. If the obfuscation was ideal, then even allowing the simulator direct access to the adversary's auxiliary input would be equivalent to the model above. Even with a weaker form of obfuscation, it still might not be possible to extract a signature from the obfuscated program even using non-black-box techniques.

7 Acknowledgements

Alper Çakan and Justin Raizes were supported by the following grants of Vipul Goyal: NSF award 1916939, DARPA SIEVE program, a gift from Ripple, a DoE NETL award, a JP Morgan Faculty Fellowship, a PNC center for financial services innovation award, and a Cylab seed funding award.

References

- [Aar09] Scott Aaronson. Quantum copy-protection and quantum money. In *2009 24th Annual IEEE Conference on Computational Complexity*, pages 229–242, 2009.
- [ABC22] Arasu Arun, Joseph Bonneau, and Jeremy Clark. Short-lived zero-knowledge proofs and signatures. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part III*, volume 13793 of *LNCS*, pages 487–516, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.

- [AC12] Scott Aaronson and Paul Christiano. Quantum money from hidden subspaces. In *Proceedings of the Forty-Fourth Annual ACM Symposium on Theory of Computing*, STOC '12, page 41–60, New York, NY, USA, 2012. Association for Computing Machinery.
- [AK24] Kasra Abbaszadeh and Jonathan Katz. Non-interactive zero-knowledge proofs with certified deletion. *IACR Cryptol. ePrint Arch.*, page 1848, 2024.
- [AKN⁺23] Shweta Agrawal, Fuyuki Kitagawa, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Public key encryption with secure key leasing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 581–610, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [APV23] Prabhanjan Ananth, Alexander Poremba, and Vinod Vaikuntanathan. Revocable cryptography from learning with errors. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 93–122, Taipei, Taiwan, November 29 – December 2, 2023. Springer, Cham, Switzerland.
- [BBBV97] Charles H Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM journal on Computing*, 26(5):1510–1523, 1997.
- [BCG⁺23] Gabrielle Beck, Arka Rai Choudhuri, Matthew Green, Abhishek Jain, and Pratyush Ranjan Tiwari. Time-deniable signatures. *Proc. Priv. Enhancing Technol.*, 2023(3):79–102, 2023.
- [BG93] Mihir Bellare and Oded Goldreich. On defining proofs of knowledge. In Ernest F. Brickell, editor, *CRYPTO'92*, volume 740 of *LNCS*, pages 390–420, Santa Barbara, CA, USA, August 16–20, 1993. Springer, Berlin, Heidelberg, Germany.
- [BGK⁺24] James Bartusek, Vipul Goyal, Dakshita Khurana, Giulio Malavolta, Justin Raizes, and Bhaskar Roberts. Software with certified deletion. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part IV*, volume 14654 of *LNCS*, pages 85–111, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [BI20] Anne Broadbent and Rabib Islam. Quantum encryption with certified deletion. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 92–122, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland.
- [BK23] James Bartusek and Dakshita Khurana. Cryptography with certified deletion. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 192–223, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [BKP18] Nir Bitansky, Yael Tauman Kalai, and Omer Paneth. Multi-collision resistance: a paradigm for keyless hash functions. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *50th ACM STOC*, pages 671–684, Los Angeles, CA, USA, June 25–29, 2018. ACM Press.
- [BR24] James Bartusek and Justin Raizes. Secret sharing with certified deletion. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22,*

- 2024, *Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 184–214. Springer, 2024.
- [BS20] Nir Bitansky and Omri Shmueli. Post-quantum zero knowledge in constant rounds. In Konstantin Makarychev, Yury Makarychev, Madhur Tulsiani, Gautam Kamath, and Julia Chuzhoy, editors, *52nd ACM STOC*, pages 269–279, Chicago, IL, USA, June 22–26, 2020. ACM Press.
 - [BS23] Shalev Ben-David and Or Sattath. Quantum tokens for digital signatures. *Quantum*, 7:901, 2023.
 - [CDNO97] Ran Canetti, Cynthia Dwork, Moni Naor, and Rafail Ostrovsky. Deniable encryption. In Burton S. Kaliski Jr., editor, *CRYPTO’97*, volume 1294 of *LNCS*, pages 90–104, Santa Barbara, CA, USA, August 17–21, 1997. Springer, Berlin, Heidelberg, Germany.
 - [CGV22] Andrea Coladangelo, Shafi Goldwasser, and Umesh V. Vazirani. Deniable encryption in a quantum world. In Stefano Leonardi and Anupam Gupta, editors, *54th ACM STOC*, pages 1378–1391, Rome, Italy, June 20–24, 2022. ACM Press.
 - [CMSZ22] Alessandro Chiesa, Fermi Ma, Nicholas Spooner, and Mark Zhandry. Post-quantum succinct arguments: Breaking the quantum rewinding barrier. In *62nd FOCS*, pages 49–58, Denver, CO, USA, February 7–10, 2022. IEEE Computer Society Press.
 - [DDN91] Danny Dolev, Cynthia Dwork, and Moni Naor. Non-malleable cryptography (extended abstract). In *23rd ACM STOC*, pages 542–552, New Orleans, LA, USA, May 6–8, 1991. ACM Press.
 - [DFMS19] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
 - [DNS98] Cynthia Dwork, Moni Naor, and Amit Sahai. Concurrent zero-knowledge. In *30th ACM STOC*, pages 409–418, Dallas, TX, USA, May 23–26, 1998. ACM Press.
 - [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer, Berlin, Heidelberg, Germany.
 - [GMR24] Vipul Goyal, Giulio Malavolta, and Justin Raizes. Unclonable commitments and proofs. In *22nd Theory of Cryptography Conference 2024, TCC 2024, Dec 2-6, 2024, Bocconi University, Milan, Italy*, 2024.
 - [GO94] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, December 1994.
 - [HKM⁺24] Taiga Hiroka, Fuyuki Kitagawa, Tomoyuki Morimae, Ryo Nishimaki, Tapas Pal, and Takashi Yamakawa. Certified everlasting secure collusion-resistant functional encryption, and more. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part III*, volume 14653 of *LNCS*, pages 434–456, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

- [HMNY22] Taiga Hiroka, Tomoyuki Morimae, Ryo Nishimaki, and Takashi Yamakawa. Certified everlasting zero-knowledge proof for QMA. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 239–268, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [HW21] Andreas Hülsing and Fiona Johanna Weber. Epochal signatures for deniable group chats. In *2021 IEEE Symposium on Security and Privacy*, pages 1677–1695, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.
- [JLLW23] Aayush Jain, Huijia Lin, Ji Luo, and Daniel Wichs. The pseudorandom oracle model and ideal obfuscation. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 233–262, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 60–73, Virtual Event, Italy, June 21–25, 2021. ACM Press.
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [JSI96] Markus Jakobsson, Kazue Sako, and Russell Impagliazzo. Designated verifier proofs and their applications. In Ueli M. Maurer, editor, *EUROCRYPT’96*, volume 1070 of *LNCS*, pages 143–154, Saragossa, Spain, May 12–16, 1996. Springer, Berlin, Heidelberg, Germany.
- [LZ19] Qipeng Liu and Mark Zhandry. Revisiting post-quantum Fiat-Shamir. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 326–355, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [MPY24] Tomoyuki Morimae, Alexander Poremba, and Takashi Yamakawa. Revocable quantum digital signatures. In Frédéric Magniez and Alex Bredariol Grilo, editors, *19th Conference on the Theory of Quantum Computation, Communication and Cryptography, TQC 2024, September 9-13, 2024, Okinawa, Japan*, volume 310 of *LIPIcs*, pages 5:1–5:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [Pas03] Rafael Pass. On deniability in the common reference string and random oracle model. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 316–337, Santa Barbara, CA, USA, August 17–21, 2003. Springer, Berlin, Heidelberg, Germany.
- [Por23] Alexander Poremba. Quantum proofs of deletion for learning with errors. In Yael Tsa-man Kalai, editor, *14th Innovations in Theoretical Computer Science Conference, ITCS 2023, January 10-13, 2023, MIT, Cambridge, Massachusetts, USA*, volume 251 of *LIPIcs*, pages 90:1–90:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.
- [Unr12] Dominique Unruh. Quantum proofs of knowledge. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 135–152, Cambridge, UK, April 15–19, 2012. Springer, Berlin, Heidelberg, Germany.

- [Unr16a] Dominique Unruh. Collapse-binding quantum commitments without random oracles. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 166–195, Hanoi, Vietnam, December 4–8, 2016. Springer, Berlin, Heidelberg, Germany.
- [Unr16b] Dominique Unruh. Computationally binding quantum commitments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 497–527, Vienna, Austria, May 8–12, 2016. Springer, Berlin, Heidelberg, Germany.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *53rd FOCS*, pages 679–687, New Brunswick, NJ, USA, October 20–23, 2012. IEEE Computer Society Press.
- [Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.